



Functional Testing of Security in Industrial Gateways under the New Machinery Regulation

Bachelor's Thesis T3_3300

Degree Programme Electrical Engineering

Field of Study Automation

Duale Hochschule Baden-Württemberg Ravensburg, Campus Friedrichshafen

by

Nathalie Nuber

Date of Submission:	19th September 2026
Timeframe:	06th July 2026 - 19th September 2026
Matriculation Number:	2257314
Course:	TEA 23
Dual Partner:	ifm ecomatic GmbH, Kressbronn a.B.
Supervisor:	M. Eng. Till Monzel
Reviewer:	M. Sc. Andreas Lau

Confidentiality Clause

The contents of this thesis may not be made available, either as a whole or in excerpts, to persons outside the examination and evaluation process, unless explicitly authorised by the Dual Partner.

Kressbronn a.B., 19th September 2026

Nathalie Nuber

Nathalie Nuber

Declaration of Authorship

I hereby declare that I have authored this thesis independently, that I have not used other than the declared sources and resources, and that this thesis has not been submitted for any other examination with identical or comparable content, nor has it been published so far.

Kressbronn a.B., 19th September 2026

Nathalie Nuber

Nathalie Nuber

Abstract

Under Regulation (EU) 2023/1230, cybersecurity has become a mandatory part of machinery conformity assessment, particularly where industrial gateways influence safety-related functions [1, p. 4]. Established functional validation does not yet systematically verify security functions. This thesis therefore develops and partially validates a systematic testing methodology for this purpose.

Following the Design Science Research Methodology (DSRM), fourteen security requirements were derived from Annex III of the Regulation via prEN 50742 to IEC 62443-4-2 Component Requirement (CR). Each requirement was linked to affected assets, a STRIDE threat, and a security function within a two-level test catalogue. An existing company test framework was integrated through a four-outcome applicability assessment and classified using the IEC 62443-4-1 verification taxonomy. The methodology was demonstrated on a Safe Remote I/O (SRIO) device under the direct-access network position (NV1), against a target component security capability of SL-C 1 as the assessment baseline for all derived requirements.

Of the ten company tests, four were applicable and executed, two were applicable with adaptation, three were not applicable, and one was blocked. Of the fourteen requirements, two were assessed as Fail due to evidenced capability deviations, and twelve remained Not Assessed because their acceptance conditions require the passive-observation or in-path network variants, an induced event, or the engineering tool.

The central finding is that a safe-state response and cybersecurity conformity are separable properties requiring independent verification. A defined safe state after a fault does not by itself demonstrate accountability, integrity, or traceability.

Kurzfassung

Gemäß der Verordnung (EU) 2023/1230 ist Cybersicherheit zu einem obligatorischen Bestandteil der Konformitätsbewertung von Maschinen geworden, insbesondere wenn industrielle Gateways sicherheitsbezogene Funktionen beeinflussen [1, S. 4]. Die etablierte funktionale Validierung überprüft Security-Funktionen bislang noch nicht systematisch. Diese Arbeit entwickelt daher eine systematische Testmethodik für diesen Zweck und validiert diese teilweise.

In Anlehnung an die DSRM wurden vierzehn Security-Anforderungen aus Anhang III der Verordnung über die prEN 50742 bis zu den IEC 62443-4-2 Component Requirements abgeleitet. Jede Anforderung wurde innerhalb eines zweistufigen Testkatalogs mit den betroffenen Assets, einer STRIDE-Bedrohung und einer Security-Funktion verknüpft. Ein bestehendes unternehmensinternes Test-Framework wurde durch eine Anwendbarkeitsbewertung mit vier möglichen Ergebnissen integriert und anhand der Verifizierungstaxonomie der IEC 62443-4-1 klassifiziert. Die Methodik wurde an einem SRIO-Gerät in der Netzwerkposition des direkten Zugriffs (NV1) demonstriert, wobei eine angestrebte Security-Fähigkeit der Komponente von SL-C 1 als Bewertungsgrundlage für alle abgeleiteten Anforderungen diente.

Von den zehn unternehmensinternen Tests waren vier anwendbar und wurden durchgeführt, zwei waren mit Anpassung anwendbar, drei waren nicht anwendbar und einer war blockiert. Von den vierzehn Anforderungen wurden zwei aufgrund nachgewiesener Abweichungen der Fähigkeiten als Fail (nicht bestanden) bewertet. Zwölf verblieben im Status Not Assessed (nicht bewertet), da ihre Akzeptanzbedingungen die Netzwerkvarianten der passiven Beobachtung oder der Zwischenschaltung, ein induziertes Ereignis oder das Engineering-Tool erfordern.

Die zentrale Erkenntnis ist, dass die Reaktion in den sicheren Zustand und die Cybersicherheitskonformität voneinander trennbare Eigenschaften sind, die eine unabhängige Verifizierung erfordern. Ein definierter sicherer Zustand nach einem Fehler belegt für sich genommen weder Zurechenbarkeit, Integrität noch Nachvollziehbarkeit.

Contents

1. Introduction	1
2. Fundamentals	3
2.1. Industrial Gateways at the IT/OT Interface	3
2.1.1. IT/OT Convergence	3
2.1.2. Industrial Gateways	5
2.1.3. Safety Communication	6
2.2. Security	7
2.2.1. Security Objectives	7
2.2.2. Assets and Attack Surfaces	8
2.2.3. Threats and Threat Modelling	9
2.3. Security Verification of Industrial Components	10
2.3.1. Functional Security Testing	11
2.3.2. Vulnerability Testing	11
2.3.3. Static and Dynamic Analysis	11
2.3.4. Robustness Testing and Fuzzing	13
2.3.5. Security Verification and Validation	13
2.4. Regulatory and Standardisation Framework	14
2.4.1. Machinery Regulation (EU) 2023/1230	14
2.4.2. prEN 50742	15
2.4.3. IEC 62443	17
3. Methodology	19
3.1. Research Design	19
3.1.1. Design Science Research Methodology	19
3.1.2. Research Questions	20
3.2. Problem Context and Scope	21
3.2.1. Initial Situation	21
3.2.2. Regulatory Applicability	21
3.2.3. Testing Constraints and Methodological Scope	22

3.3.	Requirements Analysis and Traceability	23
3.3.1.	Requirement Derivation	23
3.3.2.	Asset and Threat Mapping	24
3.3.3.	Security Functions	25
3.3.4.	Traceability Model	26
3.4.	Selection of Verification Techniques	27
3.4.1.	State of the Art	27
3.4.2.	Evaluation of Approaches	29
3.4.3.	Applicability Assessment of the Company Framework	30
3.4.4.	Selected Verification Strategy	30
3.5.	Test Case Development	31
3.5.1.	Test Development Process	31
3.5.2.	Test Specification	32
3.5.3.	Test Categorisation	33
3.6.	Security Test Environment	35
3.6.1.	NV1: Direct Access	35
3.6.2.	NV2: Passive Observation	35
3.6.3.	NV3: Active In-Path Manipulation	36
3.6.4.	Test System and Tooling	36
3.7.	Verification and Assessment Model	37
3.7.1.	Test Verdicts	37
3.7.2.	Requirement Assessment	39
3.7.3.	Evidence Model	39
4.	Implementation, Results and Discussion	41
4.1.	Scope of the Practical Demonstration	41
4.1.1.	System Under Test	41
4.1.2.	Implemented Test Environment	43
4.1.3.	Practical Coverage and Limitations	45
4.2.	Instantiation of the Methodology	45
4.2.1.	Requirement, Asset and Interface Mapping	45
4.2.2.	Applicability Assessment of the Company Framework for the SRIO	47
4.2.3.	Selected Test Cases	48
4.3.	Test Execution and Results	49
4.3.1.	Identification and Asset Discovery	50
4.3.2.	Communication Security	52
4.3.3.	Logging and Traceability	54
4.3.4.	Availability and Resilience	56
4.3.5.	Robustness	58

4.3.6. Authentication and Access Control	59
4.3.7. Firmware and Configuration Integrity	61
4.4. Requirement Assessment Results	61
4.4.1. Verification Status per Requirement	62
4.4.2. Coverage of the Practical Phase	65
4.5. Discussion of Results	65
4.5.1. Synthesis of Findings Across Test Categories	65
4.5.2. Answering the Research Questions	67
5. Summary and Outlook	69
5.1. Summary	69
5.2. Outlook	71
Bibliography	73
List of Abbreviations	82
List of Figures	86
List of Tables	87
A. Note on the Use of Artificial Intelligence	88
B. Detailed Requirement Derivation Matrices	89
B.1. Security Requirements Derived from Regulation (EU) 2023/1230	89
B.2. Mapping to prEN 50742	92
B.3. Mapping to International Electrotechnical Commission (IEC) 62443-4-2	95
C. Secure Development Lifecycle	100
D. Tool Commands	103
D.1. Attack-Surface Enumeration	103
D.2. Identification-Surface Inventory	103
D.3. Cleartext Transmission of the Identification Data	104
D.4. Intervention Evidence and Time Attribution	104
D.5. Availability under Connection Load	104
D.6. Layer-Two and Network Flooding	105
D.7. Robustness under Malformed Input	105
D.8. Access Control and Configuration Interfaces	106

1. Introduction

Industrial automation systems increasingly connect their Operational Technology (OT) to Information Technology (IT) networks [2, pp. 2, 9]. This convergence turns components at the boundary between the two domains into critical assets. Industrial gateways occupy exactly this boundary, translating and forwarding data between fieldbus protocols and higher-level services. Consequently, they expose previously closed OT environments to threats originating in connected IT networks [3, p. 199] [4, p. 33].

Before the new Regulation, gateway validation focused on functional behaviour and safety. Cybersecurity was not mandatory. However, Regulation (EU) 2023/1230 introduces binding cybersecurity requirements, treating protection against malicious third parties as a prerequisite for machinery safety [1, p. 5]. Functional safety can therefore no longer be evaluated independently of cybersecurity [5]. For gateways influencing safety-related functions, security properties become vital evidence for conformity assessments [6, pp. 1]. This Regulation is complemented by the draft standard prEN 50742 and the IEC 62443 series, which define component-level security requirements for Industrial Control Systems (ICSs) [7, 8].

Although the regulatory expectations are now explicit, the established validation process does not yet meet them. Existing functional and safety validation verifies the intended behaviour of a device, but it does not systematically verify its security functions. This creates a gap between the regulatory expectations on the one hand and the currently established methodology for functional testing on the other. The central problem addressed in this thesis is therefore the absence of a systematic approach for the functional testing of security in industrial gateways at the physical IT/OT interface.

The task of this thesis is to address this gap for a representative device, a Safe Remote I/O (SRIO) device. The System Under Test (SUT) is assessed as a safety-related component integrated into a machine. Its requirements are therefore derived component-level requirements. The primary objective is to research security testing approaches, frameworks, and organisational practices, and to develop a systematic testing methodology

that verifies the security functions of such a gateway and maintains traceability between machine-level requirements and concrete test activities. The scientific contribution is the systematic derivation, justification, and partial validation of this methodology, rather than the exhaustive execution of every possible test. The methodology is demonstrated on a single device in an isolated laboratory. Within this demonstration, the practical validation is limited to the direct-access position and to a subset of the specified tests. The work therefore provides a partial empirical demonstration rather than a complete conformity assessment.

The work is guided by three research questions. First, how can the machine-level cybersecurity obligations of Regulation (EU) 2023/1230 be translated into verifiable, derived component-level security tests for a safety-related industrial gateway? Second, which tests of the existing security company framework are applicable to an industrial gateway, and how can non-applicable tests be identified before execution? Third, what can direct-access testing at the physical IT/OT interface verify about the effectiveness of the implemented security functions, and where are its boundaries?

Methodologically, the thesis follows Design Science Research Methodology (DSRM) to create and evaluate a testing methodology artefact [9, p. 54]. This methodology derives security tests in three stages. It extracts machine-level obligations from the Machinery Regulation, maps them via prEN 50742 to IEC 62443-4-2 component requirements, and links them to assets and threats using Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege (STRIDE) principles. It then defines a test catalogue, selects techniques, and integrates an existing company framework where applicable. Demonstrated on a real safety gateway, the results yield a traceable methodology that facilitates the practical application of regulatory requirements in manufacturing and supplies necessary evidence for technical documentation.

The thesis is structured as follows. Chapter 2 establishes the technical, regulatory, and methodological foundations. Chapter 3 develops the testing methodology, detailing requirement derivation, traceability, and verification models. Chapter 4 applies this methodology to the SRIO device, reports the evidence, and evaluates the approach against the thesis objective. Finally, Chapter 5 summarises the findings and provides an outlook on future work.

2. Fundamentals

This chapter establishes the theoretical foundation for a systematic security-testing methodology for industrial gateways. It first characterises the industrial gateway and its safety communication at the Input/Output (I/O) interface. It then introduces the security fundamentals required to reason about such a device, namely the security objectives, assets, attack surfaces, and the threat model. Building on these concepts, the chapter outlines the verification of industrial components, including the security functions under test, the applicable testing techniques, and the IEC 62443 verification framework. It concludes by examining the regulatory and standardisation framework and the implications for the methodology.

2.1. Industrial Gateways at the IT/OT Interface

The test object of this thesis is an industrial gateway, the component at the boundary between the IT and OT domains. This section characterises this boundary, the role of the gateway, and the safety communication that constrains any security measure applied to the device.

2.1.1. IT/OT Convergence

Traditionally, OT environments, comprising the hardware and software used to monitor and control physical processes, operated as isolated, proprietary networks. These systems were managed entirely independently of IT networks, which handle the electronic storage, calculation, and transport of business data [2, p. 2] [3, p. 199].

However, the rise of Industry 4.0 has transformed these isolated environments into interconnected architectures [10, pp. 14]. IT/OT convergence defines this strategic integration,

where systems designed for monitoring and controlling physical machinery are connected with data-centric computing systems [2, p. 2]. The primary driver for this convergence is the necessity to create a single, synergised view of enterprise operations, ensuring that the right information flows from a physical sensor to a digital application in real-time [2, p. 4].

To systematically understand how this information flow is organised, industrial architectures can be categorised into four functional levels [11]:

1. **Process Level:** Physical sensors and actuators interacting directly with the environment to perform mechanical work.
2. **Control Level:** Programmable Logic Controllers (PLCs) executing process logic and regulating physical operations in real-time.
3. **Supervisory/Operation Level:** Human Machine Interfaces (HMIs) and Supervisory Control and Data Acquisition (SCADA) systems allowing operators to monitor states and intervene manually.
4. **Operation IT / Data Layer:** The aggregation layer where raw OT data is structured and forwarded to higher-level IT applications, such as Enterprise Resource Planning (ERP) or Customer Relationship Management (CRM) systems [12, p. 404].

This Data Layer represents the critical intersection where operational systems become direct extensions of information systems [2, p. 4]. Converging these two domains yields substantial economic advantages, primarily through enhanced process transparency, predictive maintenance strategies, and data-driven optimisation [11].

Despite these benefits, the physical and software integration of legacy OT equipment with modern IT networks introduces significant engineering and organisational friction [13]. This friction arises because both domains operate under fundamentally conflicting paradigms. Table 2.1 summarises these core differences.

Table 2.1.: Differences between IT and OT environments (Based on [11] [3, p. 1, 199])

Characteristic	IT	OT
Primary Focus	Information processing and data flow	Monitoring and control of physical processes
Security Priorities	Confidentiality → Integrity → Availability (CIA)	Availability → Integrity → Confidentiality (AIC)
Core Security Objectives	Authentication, Non-repudiation	Real-time availability, Functional Safety
Protocol Examples	TCP/IP, HTTP(S)	Modbus, PROFINET
Update Cycles	Regular, frequent, and auto-mated	Rare, conservative, and strictly validated

The most critical divergence lies in the prioritisation of security objectives. In IT paradigms, data confidentiality and privacy occupy the highest priority to prevent unauthorised access to sensitive information. In contrast, OT environments operate under the Availability, Integrity and Confidentiality (AIC) triad, prioritising availability and deterministic execution above all else [3, p. 199]. Any latency or system interruption on the shop floor could lead to physical damage, severe safety hazards, or costly production downtime [3, p. 1] [11].

This reversed priority directly dictates the motivation behind differing lifecycle and maintenance strategies. IT infrastructures rely on frequent, rapid security patching to close vulnerabilities as quickly as possible. OT operators, however, view changes to a stable system as risky and costly [13]. Because industrial controllers require rigorous validation to guarantee operational safety, operators fear that software updates might induce unexpected behaviour or system crashes. Consequently, patches are applied rarely and strictly during planned maintenance windows. Furthermore, the underlying network communication in OT frequently utilises legacy, low-overhead protocols that lack built-in security features such as encryption, significantly exacerbating the risk at the IT/OT interface once these networks are converged [11].

2.1.2. Industrial Gateways

Industrial gateways bridge distinct network environments. They enable communication between IT and OT systems [14].

To achieve this, a gateway intercepts incoming data from a source system, translates or encapsulates the payload into a compatible format, and routes it to the target system. During this conversion process, protocol-specific overhead that is incompatible with the destination network is stripped away, while necessary parameters required by the target protocol are dynamically appended [15, p. 277]. Beyond complex protocol translation, gateways can integrate raw binary or analogue signals from basic physical sensors into bus systems for further network processing [10, p. 132][16] .

A gateway can operate across various layers of the Open Systems Interconnection (OSI) reference model [14]. Industrial gateways mediate between IT and OT networks that do not follow a uniform layering. Rather than confining OT to the lower layers, many industrial protocols compress the OSI stack, omitting or thinning layers 3 to 6 to achieve deterministic, low-latency communication, while still exposing application-layer services. Consequently, managing this transition requires the gateway to reconcile various domain-specific constraints. These include differing physical layer standards, addressing schemes, transmission speeds, and data formats [14].

To ensure efficient translation, the protocol mapping generally occurs at the lowest common layer shared by the interconnecting systems. Depending on the specific OSI layer at which the gateway operates, its internal processing mechanism and performance characteristics vary significantly [17].

2.1.3. Safety Communication

A safety industrial gateway transmits safety-related data over Process Field Network (PROFINET) using the PROFIsafe profile [18, p. 439]. PROFIsafe follows the black-channel principle. The safety layer does not trust the underlying transmission channel and instead protects each safety telegram from producer to consumer, so that the standard communication stack and the network components are not part of the safety function [19].

Each safety telegram is protected by several measures. A consecutive number orders the telegrams and detects loss, repetition, or insertion. A Cyclic Redundancy Check (CRC) protects the payload against corruption. A safety destination address binds a telegram to its intended consumer. A watchdog time bounds the maximum age of a valid telegram. When it expires, the consumer passivates to the defined safe state [20].

2.2. Security

Cybersecurity comprises the processes and technical measures used to protect systems, networks, and data from digital threats, with the aim of preventing unauthorised access, data breaches, and operational disruption [21] [22]. For an industrial gateway, this protection must extend beyond the IT network to the cyber-physical function of the device [23, p. 17]. The following subsections introduce the security objectives, the assets and attack surfaces, and the threat model required to reason about the security of such a device.

2.2.1. Security Objectives

Information security is traditionally structured around the Confidentiality, Integrity and Availability (CIA) triad [24] [25, p. 1]. This paradigm conceptualises the three primary objectives required to establish a security posture and protect information assets against digital threats [26, p. 5]. To evaluate and mitigate risks effectively, these three distinct dimensions form the theoretical baseline [25, p. 24]:

- **Confidentiality:** Focuses on ensuring that sensitive data and information are strictly protected from unauthorised access or disclosure to unintended individuals, entities, and processes [26, p. 4] [25, p. 3]. In the specific context of Industrial Automation and Control Systems (IACS), the maintenance of authorised restrictions on access becomes paramount to secure proprietary information, data configurations, and core operational parameters from illicit exposure [27, p. 23].
- **Integrity:** The assurance that information and system components remain accurate, reliable, and trustworthy throughout their entire lifecycle [26, p. 4] [25, p. 3]. This objective actively safeguards digital assets against accidental or unauthorised modifications, deletions, and fabrications, which prevents erroneous decision-making or potential operational harm caused by altered data [26, pp. 4].
- **Availability:** Guarantees that essential services, computing systems, and communication interfaces remain continuously accessible and operational for authorised users precisely when needed [26, pp. 4] [25, p. 3]. This dimension dictates the overall resilience of the architecture against malicious disruptions, ensuring that operations can be sustained or recovered quickly to maintain customer trust and minimise financial losses [26, p. 5].

In practice, these primary goals do not operate in isolation. They often require deliberate design trade-offs to balance conflicting requirements. While the CIA triad establishes a vital baseline, modern connected environments demand explicit extensions to address sophisticated threats. Key additions include authenticity, non-repudiation, and accountability [28, pp. 102] [24]:

- **Authenticity:** Ensures that information and assets are genuine, verifiable, and trustworthy. It confirms that data originates from the expected sender [28, p. 102].
- **Non-Repudiation:** Prevents an entity from falsely denying having performed an action or sent a message. It establishes trust and legal responsibility by uniquely identifying the sender of safety-critical data [28, pp. 102].
- **Accountability:** Guarantees that actions or information can be uniquely traced back to the responsible entity. It is often enforced through enterprise-level policies, such as IEC 62443, which assign specific roles and obligations [28, p. 103].

2.2.2. Assets and Attack Surfaces

In the context of cybersecurity, an **asset** represents any physical object, logical artefact, or information that holds value to an organisation [28, p. 85] [29, p. 5]. The compromise of an asset's cybersecurity properties can lead to significant damage scenarios [28, p. 85]. A comprehensive cybersecurity assessment requires a granular decomposition of a system into three primary asset categories:

- **Hardware Assets:** This category encompasses physical components such as micro-controllers, Printed Circuit Boards (PCBs), memory blocks, and physical communication interfaces.
- **Software Assets:** This dimension includes the operating system, bootloaders, embedded firmware, communication protocol stacks, and specific application logic.
- **Data Assets:** This group consists of sensitive information stored or processed by the system. Examples include configuration files, calibration data, cryptographic keys, and user credentials [28, pp. 85] [29, p. 5] [30, p. 3].

Every asset has specific cybersecurity properties, primarily based on the CIA triad, which

define its individual protection needs [29, p. 5].

The **attack surface** of a system is defined as the union of all accessible code, interfaces, services, and protocols. It represents the sum of all externally visible actions and resources where unauthorised users can inject malicious inputs or attempt to compromise the system [31, p. 1] [32, p. 80]. Analysing this surface is the starting point for mapping potential threats to the identified assets [32, p. 80].

2.2.3. Threats and Threat Modelling

A **threat** is a circumstance or event that can adversely affect an organisation's operations, assets, or individuals, for example through unauthorised data access, destruction, or Denial of Service (DoS) [27, p. 23]. A **vulnerability** is a flaw or weakness in the design, implementation, or operation of a system [7, p. 12]. Its exploitation compromises the protective measures, and therefore a vulnerability must be eliminated, mitigated, or covered by a documented compensating countermeasure [7, p. 9] [7, p. 14]. An **attack vector** is the path an adversary uses to exploit a vulnerability, and it connects a theoretical threat to the practical compromise of an asset [32, p. 80] [33]. Systematic threat modelling therefore examines every hardware, software, and data asset for the vectors that could reach it [30, p. 2, p. 11].

Threat modelling is framed by Threat Analysis and Risk Assessment (TARA), a systematic method that identifies threats, assesses their risk, and derives countermeasures [28, p. 37]. TARA differs from the safety-oriented Hazard Analysis and Risk Assessment because it assumes an intentional adversary rather than a random fault, and it is reassessed as the threat landscape evolves [29, p. 70].

STRIDE is the threat-modelling instrument. Developed by Microsoft, it identifies threats from the perspective of attacker goals rather than as a static list [34] [28, p. 143] [35]. Its acronym denotes six threat categories:

- **Spoofing:** An attacker assumes a false identity, for example by using another entity's credentials or by impersonating a component in an embedded network.
- **Tampering:** Unauthorised modification of data or functionality, such as altering stored data, firmware, or data in transit.

- **Repudiation:** The ability to deny an action when the system keeps insufficient audit records to prove it.
- **Information Disclosure:** Exposure of data to unauthorised parties, including reading data in transit.
- **Denial of Service:** Disruption of operation or denial of access to legitimate users, for example by flooding an interface.
- **Elevation of Privilege:** An unprivileged attacker gains higher access rights than authorised.

Each category is the direct negation of a security objective, as Table 2.2 shows [28, p. 144] [35].

Table 2.2.: Mapping of STRIDE threat categories to cybersecurity properties (Based on [28, p. 144] [35])

Threat Category	Cybersecurity Property Violated
Spoofing	Authenticity
Tampering	Integrity
Repudiation	Non-repudiation, Accountability
Information Disclosure	Confidentiality
Denial of Service	Availability
Elevation of Privilege	Authorisation

2.3. Security Verification of Industrial Components

Security is often treated as a non-functional requirement, alongside performance and usability [36, p. 291]. For a security-centric component, however, the security mechanisms are core functional requirements and must be verified as such [37, p. 61]. This section presents the techniques used to verify the security functions of an industrial gateway, and the IEC 62443 verification taxonomy that classifies these techniques.

A security test also differs in purpose from a safety test. Functional safety validation examines the behaviour of a device under random faults and does not assume an adversary. Security verification instead assumes a deliberate attacker and applies a security-specific acceptance criterion [29, p. 70]. An existing safety test therefore does not, on its own, demonstrate a security property, even when it exercises the same interface.

2.3.1. Functional Security Testing

Functional security testing verifies that an explicitly planned security function operates as specified [38, p. 82] [32, p. 8]. Its test cases are derived from the security requirements, in the same way that use-case-based functional testing derives cases from functional requirements [39, p. 56].

The available knowledge of the internal structure determines the test perspective [36, p. 296]. In black-box testing the internal structure is unknown and only the external interfaces are exercised; in white-box testing the source code is available; grey-box testing combines both.

2.3.2. Vulnerability Testing

Whereas functional testing confirms a planned function, vulnerability testing searches for unaddressed weaknesses [32, p. 8]. Vulnerability scanning uses automated tools to enumerate hosts, open ports, and services and to match them against databases of known weaknesses [40, p. 27]. Penetration testing goes further and mimics an attacker to identify and exploit weaknesses, following the four stages of planning, discovery, attack, and reporting defined in National Institute of Standards and Technology (NIST) SP 800-115 [40, p. 37].

2.3.3. Static and Dynamic Analysis

Security analysis techniques are structured into static and dynamic execution paradigms. Static analysis inspects source code or control and data flow graphs using checklists without exercising the system with real runtime data [32, p. 35]. It detects errors early in development, but may miss runtime flaws [41, p. 1]. Conversely, dynamic analysis evaluates system behaviour during runtime under different configurations and input data, comparing actual outputs against formal specifications [32, p. 35]. Dynamic testing provides real-world validation, but it can disrupt system operations [41, p. 1].

Static Analysis Mechanisms:

Static Application Security Testing (SAST) examines source code or binaries without

execution [42]. As shown in Table 2.3, static detection primarily relies on code similarity-based and code pattern-based approaches [41, pp. 3]:

- **Code Similarity-Based Detection:** Identifies potential vulnerabilities by searching for known vulnerable code clones via string comparison, syntax analysis, or machine learning [41, p. 3]. Grammar-based techniques analyse grammatical code structures, whereas semantic-based techniques evaluate control and data flows to determine code meaning [41, pp. 4]. This approach requires less training data but cannot detect novel vulnerabilities [41, p. 8].
- **Code Pattern-Based Detection:** Utilises machine learning and deep learning during a training and detection phase to recognise specific vulnerable code patterns [41, p. 5]. While dependent on data, it can discover new vulnerabilities [41, p. 8].

Combining both static approaches increases overall vulnerability detection effectiveness [41, p. 8].

Table 2.3.: Comparison of static analysis detection approaches (Adapted from [41, p. 3, 8])

Property	Code Similarity-Based	Code Pattern-Based
Operating Principle	Searches for similar code clones	Identifies specific code patterns
Techniques Used	String comparison, syntax and semantic analysis	Machine learning, deep learning
Advantages	Less dependent on training data	Capable of detecting new vulnerabilities
Disadvantages	Unable to detect new vulnerabilities	Highly dependent on training data

Dynamic Analysis Mechanisms:

Dynamic Application Security Testing (DAST) evaluates running applications under real-world attack conditions [42]. Advanced dynamic penetration testing methodologies employ planning-based or reinforcement learning-based approaches to discover execution paths [41, p. 6]. Furthermore, Interactive Application Security Testing (IAST) combines static and dynamic feedback during runtime, while Software Composition Analysis (SCA) audits third-party dependencies for vulnerabilities and licence compliance [42].

2.3.4. Robustness Testing and Fuzzing

Robustness is the ability of a system to continue operating correctly under exceptional inputs or environmental stress [32, p. 39]. Fuzzing is an automated robustness technique that repeatedly injects invalid, random, or unexpected inputs into an interface to provoke faults or crashes. A dumb fuzzer mutates inputs without knowledge of their format, whereas a smart fuzzer uses an input model; depending on the available access, fuzzing is further divided into white-box, grey-box, and black-box approaches [41, p. 8] [32, p. 29].

2.3.5. Security Verification and Validation

IEC 62443-4-1 defines Security Verification and Validation Testing (SVV) as the practice that demonstrates that a component meets its security requirements and its intended security purpose. Verification testing shows that the requirements are met, whereas validation additionally addresses the product security context [43, p. 16]. This practice encompasses five specific requirements. While the first four define distinct test types, the fifth governs the organisational execution of these tests [43, pp. 37]:

- **SVV-1 (Security Requirements Testing):** Verifies that the security functions meet the requirements and that the product handles error cases and invalid input correctly, including functional, performance, and boundary tests [43, p. 38].
- **SVV-2 (Threat Mitigation Testing):** Tests the effectiveness of the mitigations in the threat model, including deliberate attempts to defeat each mitigation, using the STRIDE categories as example threats [43, pp. 38].
- **SVV-3 (Vulnerability Testing):** Identifies potential vulnerabilities through abuse-case and fuzz testing, attack-surface analysis, and scans for known vulnerabilities [43, p. 39].
- **SVV-4 (Penetration Testing):** Attempts to discover and exploit vulnerabilities from an attacker's perspective, often by chaining several weaknesses [43, pp. 39].
- **SVV-5 (Independence of Testers):** Grades the required level of independence of the testers from the development team, based on the targeted security level [43, pp. 40].

2.4. Regulatory and Standardisation Framework

2.4.1. Machinery Regulation (EU) 2023/1230

With the publication of Regulation (EU) 2023/1230 (hereafter 'the Regulation') in the Official Journal of the European Union on 29 June 2023, the European legislature initiated a fundamental paradigm shift regarding the product safety of machinery [44, p. 32]. The new Regulation replaces the former Machinery Directive 2006/42/EC and becomes mandatory from 20 January 2027, following a transition period that ends the preceding day [1, p. 38].

The primary legal distinction lies in its regulatory form. While the previous directive was addressed strictly to EU member states, requiring transposition into national law, a regulation applies directly to all economic operators, including manufacturers, importers, and distributors [44, p. 32] [1, p. 15]. This creates a standardised, uniform legal framework across the Union without national leeway for interpretation [1, p. 38]. Furthermore, the Regulation introduces legally binding definitions for fundamental concepts such as “substantial modification”, formally clarifying when an alteration to an existing machine legally turns an operator into a manufacturer subject to full conformity assessment procedures [1, p. 25].

A key driver for this legislative overhaul was the identification of regulatory gaps within the previous directive regarding emerging digital technologies [1, p. 2]. Consequently, the Regulation incorporates mandatory requirements for cybersecurity and protection against malicious third parties into the framework of product safety for the first time [44, p. 33]. Under the Regulation, cybersecurity is explicitly treated as a prerequisite for machinery safety, establishing that the functional safety of modern machinery can no longer be evaluated independently of its cybersecurity [5].

This digitalisation shift is also reflected in structural updates across the Regulation's annexes. Most notably, the explicit list of safety components in Annex II now formally includes standalone software that fulfils safety functions. This establishes that non-tangible, digital components are subject to the same strict conformity requirements as physical safety hardware [1, pp. 42].

Specific cybersecurity requirements apply to machinery and related products incorporating such components under Regulation (EU) 2023/1230 [1, p. 44]. Annex III, Section 1.1.9 formulates explicit provisions regarding protection against corruption [1, pp. 47]:

- The connection of any external device to the machine must not lead to a hazardous situation through any function of the connected device or via a remote access device [1, p. 48].
- Hardware components transmitting signals or data relevant to safety-related software must be adequately protected against accidental or intentional corruption.
- Software and data essential for compliance with safety requirements must be identified as such and adequately protected against corruption.
- The machine must collect evidence of legitimate or illegitimate interventions in the relevant hardware, software, or its configuration [1, p. 49].

Complementarily, Annex III, Section 1.2.1 addresses the safety and reliability of control systems. It requires them to be designed to withstand reasonably foreseeable malicious attempts by third parties that could lead to hazardous situations. A failure or disruption in data transmission must not cause a hazardous situation [1, p. 49].

Furthermore, Article 10(2) [1, p. 18] and Annex IV Part A [1, pp. 83] obligate manufacturers to maintain comprehensive technical documentation. This documentation must include risk assessments, descriptions of implemented protective measures, as well as test reports and trial results proving actual conformity with safety requirements [1, p. 83].

2.4.2. prEN 50742

While the Regulation defines the overarching regulatory safety objectives regarding cybersecurity, the draft standard prEN 50742, intended for harmonisation (published in Germany as DIN European Standard (EN) 50742 / VDE 0113-742), provides the specific technical guidance for engineering and implementation. The standard acts as a bridge between regulatory compliance and practical machine design, with its primary focus on fulfilling the essential health and safety requirements of the Regulation, specifically Annex III, sections 1.1.9 and 1.2.1 [7, p. 70].

prEN 50742 aims at protecting the machine's functional safety and control systems against digital and physical manipulation, ensuring that cyber threats cannot cause hazardous situations [7, p. 10]. A fundamental requirement of the standard is that any implemented cybersecurity measure must not compromise or degrade the functional safety of the machine,

for instance, by introducing impermissible latencies in safety-critical communication loops [7, p. 14].

To accommodate different organisational prerequisites and product architectures, the standard defines two distinct conformity paths for manufacturers [7, p. 15]:

- **Approach A** represents a standalone, machine-specific methodology directly linked to classical safety risk assessments according to EN ISO 12100 [7, p. 16]. Under this approach, the required strength of countermeasures is determined by calculating the Safety Related Security Level (SRSL), 0 to 3, which is derived from the attack potential combined with the severity of potential safety-related harm [7, p. 19].
- **Approach B** is specifically designed for complex systems and components integrated into larger industrial communication networks. Instead of using the standalone SRSL framework, Approach B utilises the established EN IEC 62443 series to define and verify technical security requirements [7, p. 16, 24]. Under this approach, the system as a whole must comply with EN IEC 62443-3-3, while individual machine components must meet the technical security requirements of EN IEC 62443-4-2 at a specified Security Level Capability (SL-C) (typically SL-C 1 or SL-C 2) [7, p. 24].

Despite methodological differences, both paths share identical core objectives regarding the technical protection of interfaces and the verification of control system integrity [7, p. 15]. Both frameworks mandate critical mitigation measures. Entities attempting to intervene must be authenticated, and logical access must be restricted through role-based authorisation [7, p. 17, 24]. The integrity of safety-related software and configuration data must be verified cryptographically [7, pp. 17].

Furthermore, exchanged information across network boundaries must be secured to prevent spoofing or tampering [7, p. 50]. Both approaches require secure, tamper-resistant digital logging of security-relevant interventions; the tracing log must retain at least the records of the last intervention of each type for a minimum of five years to support traceability [7, p. 19, 23]. Furthermore, prEN 50742 establishes that cybersecurity compliance cannot be achieved by design alone. All implemented technical countermeasures must be structurally verified [7, p. 14].

2.4.3. IEC 62443

The IEC 62443 series is the reference framework for securing IACS [27, p. 13].

At the system level, IEC 62443-3-3 organises its requirements around seven Foundational Requirements (FRs). These requirements structure the Component Requirements (CRs) that follow and are therefore reproduced here [27, p. 15]:

- **FR 1 (Identification and Authentication Control):** Unique identification and authentication of all entities before access is granted [27, p. 29].
- **FR 2 (Use Control):** Restriction of privileges according to the principle of least privilege [27, p. 41].
- **FR 3 (System Integrity):** Protection of the system, applications, and data against unauthorised modification [27, p. 50].
- **FR 4 (Data Confidentiality):** Protection of sensitive information against unauthorised disclosure [27, p. 57].
- **FR 5 (Restricted Data Flow):** Network segmentation to limit data flow and lateral movement [27, p. 61].
- **FR 6 (Timely Response to Events):** Security monitoring and logging for incident response [27, p. 65].
- **FR 7 (Resource Availability):** Continued operation under resource stress or DoS [27, p. 67].

The fulfilment of these requirements is graded through Security Levels (SLs) (SL 1 to SL 4), which express the resilience against increasing adversary profiles [27, p. 16, 26].

IEC 62443-4-1 specifies the process-oriented requirements for a Secure Development Life Cycle (SDLC) [43, p. 8]. It defines eight practice areas, from Security Management to Security Update Management, and grades organisational rigour through Maturity Levels (MLs) (ML 1 to ML 4) [43, pp. 20].

IEC 62443-4-2 specifies the technical security requirements for IACS components [8, p.

18]. It maps the seven FRs onto four component types, namely software applications, embedded devices, host devices, and network devices [8, p. 17]. The requirements are expressed as CRs and refined by Requirement Enhancements (REs) to reach a Component SL-C [8, pp. 31]. For example, the system-level restricted-data-flow requirement of FR 5 becomes an explicit component capability, such as the ability of a network component to enforce logical isolation at its interfaces [8, p. 61].

3. Methodology

Building on the theoretical foundations established in Chapter 2, this chapter develops the security-testing methodology used in this thesis. The methodology translates the machine-level cybersecurity obligations of Regulation (EU) 2023/1230 into derived, component-level requirements and a traceable, repeatable engineering procedure. The chapter follows the derivation chain that forms the scientific core of the methodology.

Section 3.1 defines the research design, the research questions, and the constraints. Section 3.2 describes the initial situation, the regulatory allocation of responsibility, and the methodological scope. Section 3.3 explains how the fourteen security requirements are derived from the regulation and how each requirement is mapped to the affected assets, the relevant threats, and the security functions that address them. These links are consolidated into a single traceability chain. Section 3.4 selects the verification techniques and assesses the applicability of the company framework. Section 3.5 defines the development of the individual test cases. Section 3.6 defines the test environment and the three network access variants. Section 3.7 defines the verification and assessment model that converts test outcomes into requirement verdicts.

3.1. Research Design

This section defines the research method and the research questions that bound the methodology.

3.1.1. Design Science Research Methodology

This research follows DSRM, proposed by Peffers et al. [9, p. 46]. DSRM structures the creation and evaluation of an artefact that solves a practical engineering problem. The

artefact of this thesis is a standards-based methodology for the functional security testing of an industrial gateway. The six activities of the DSRM process map onto the structure of this thesis:

1. **Problem Identification and Motivation:** The absence of a systematic approach for verifying security functions derived from Regulation (EU) 2023/1230 is established in Chapter 1.
2. **Definition of Objectives:** The objective of a traceable, standards-based verification methodology that uses prEN 50742 and the IEC 62443 series is defined in Chapter 1.
3. **Design and Development:** The methodology, its environment, and its governance are designed in this chapter.
4. **Demonstration:** The methodology is applied to a physical SUT in Chapter 4.
5. **Evaluation:** The results and the residual validation gap are assessed in Chapter 4.
6. **Communication:** The results are documented and discussed in this thesis.

3.1.2. Research Questions

The design and evaluation of the artefact are guided by three research questions:

- **RQ1:** How can the machine-level cybersecurity obligations of Regulation (EU) 2023/1230 be translated into verifiable, derived component-level security tests for a safety-related industrial gateway?
- **RQ2:** Which tests of the existing security company framework are applicable to an industrial gateway, and how can non-applicable tests be identified before execution?
- **RQ3:** What can direct-access testing at the physical IT/OT interface verify about the effectiveness of the implemented security functions, and where are its boundaries?

3.2. Problem Context and Scope

This section describes the initial situation and the subject area of the SUT, clarifies how Regulation (EU) 2023/1230 applies to it and how responsibility is allocated between component and machine, and defines the constraints and the scope that bound the methodology.

3.2.1. Initial Situation

The SUT is a SRIO device. It exposes two interfaces with different characteristics. The first interface carries deterministic safety communication over PROFINET and PROFIsafe according to the black-channel principle. The second interface is a diagnostic service reachable over Hypertext Transfer Protocol (HTTP). The device therefore combines an OT fieldbus path and an IT diagnostic path in a single component. Ifm ecomatic operates an established functional test process. This process does not yet verify security functions in a systematic and requirement-driven manner. The methodology developed here addresses this gap by adding a systematic, requirement-driven security verification. It complements the existing functional and safety validation rather than replacing it.

3.2.2. Regulatory Applicability

Regulation (EU) 2023/1230 addresses machinery, related products, safety components and partly completed machinery [1, pp. 11]. The SUT is treated in this thesis as a safety-related component integrated into a machine, not as a machine in its own right. Its regulatory relevance therefore arises primarily through that integration. The following distinction separates the legal obligation from the engineering activities needed to support the conformity assessment of the overall machine.

- **Legal requirements on the machine:** The machine manufacturer must satisfy the essential health and safety requirements of Annex III for the machinery or related product [1, p. 44].
- **Derived technical requirements on the component:** The gateway or component must provide the capabilities derived from the machine-level obligations and inherited

through integration. These requirements are not themselves direct legal obligations imposed on the component.

- **Contractual and technical requirements on the component manufacturer:** Customer, integrator and internal product specifications may impose requirements on the component manufacturer independently of the Regulation itself.
- **Evidence required by the machine integrator:** The component manufacturer must provide the information and test evidence that the machine manufacturer or integrator requires to complete the conformity assessment of the overall machine [1, p. 20, pp. 97].

A machine-level obligation may also be satisfied by compensating countermeasures at system or zone level rather than by the component itself. Consequently, a negative component-level finding indicates an integration requirement that the integrator must document [8, p. 30] [27, pp. 27]. It does not, by itself, establish non-conformity of the product. This allocation bounds the claims of the methodology and frames the requirement assessment reported in Chapter 4.

3.2.3. Testing Constraints and Methodological Scope

Several constraints shape the methodology and are treated as fixed influencing factors:

- **Black-box target:** The SUT is evaluated as a compiled product. Its source code is not available, which excludes static source analysis.
- **Isolated laboratory:** Testing is performed in a dedicated laboratory. Production systems are excluded [45, p. 116] [46, p. 242].
- **Safety priority:** The safety function must not be degraded by testing. A cybersecurity measure or a test must never introduce impermissible latency into the safety communication [7, p. 14].
- **Limited practical time:** The practical test phase is time-limited. This affects the number of executed tests.

The scope follows from these constraints. The methodology focuses on functional security

testing, as distinguished from vulnerability testing in Section 2.3. It verifies whether planned security functions, such as access restrictions or integrity checks, operate as specified [38, p. 82]. Requirement-driven robustness and availability testing is included where a requirement demands it [7, p. 69]. This bounded robustness testing is derived from a requirement and differs from open-ended fuzzing that only searches for unknown defects.

Three activities lie outside the scope. Open-ended vulnerability discovery, such as unstructured penetration testing whose goal is to find unknown defects, is excluded. It cannot systematically demonstrate the presence and correct operation of a mandated security function. Static source-code analysis is excluded because no source code is available. Invasive hardware analysis, such as chip-level or side-channel attacks, is excluded because it exceeds a functional verification at the IT/OT interface.

3.3. Requirements Analysis and Traceability

This section defines how the machine-level obligations of Regulation (EU) 2023/1230 are analysed into verifiable, component-level requirements.

3.3.1. Requirement Derivation

The derivation follows three steps. First, machine-level cybersecurity requirements are extracted from Regulation (EU) 2023/1230. This step isolates specific obligations from general legal text and yields fourteen derived requirements documented in Appendix B.1. Second, each requirement is mapped to the draft standard prEN 50742, which provides technical guidance for machine design. The resulting mapping is documented in Appendix B.2. Third, each derived requirement is translated into CRs of IEC 62443-4-2. This step produces granular, component-level specifications for the capabilities that an integrated industrial component must provide, as documented in Appendix B.3.

This thesis follows Approach B of prEN 50742. The choice reflects the role of the SUT as a component integrated into a larger industrial communication network. The methodology therefore uses IEC 62443-4-2 CRs as the technical basis for the derived component capabilities rather than deriving an independent SRSL.

To evaluate these capabilities, the methodology establishes SL-C 1 as the target baseline for all derived requirements. Because the SUT is evaluated as an isolated component outside a specific machine context, a comprehensive, machine-level risk assessment cannot be performed. SL-C 1 is therefore selected as the pragmatic minimum standard. It represents the foundational capability required to protect against casual or coincidental violation. Consequently, a Fail verdict in this methodology indicates that the component lacks basic security standards and fails to meet the minimum baseline required for secure integration.

3.3.2. Asset and Threat Mapping

A standard clause cannot be tested without a concrete target. The methodology therefore adds a mandatory asset-mapping step. The SUT is decomposed into its physical and logical assets, for example hardware interfaces, communication services, and stored configuration data. Each requirement is paired with the assets to which it applies. This step defines not only what must be verified, but also where the corresponding security function is located. The asset mapping can also reveal additional CRs. A requirement may apply to an asset that introduces its own component-level controls, which the direct mapping from the regulation through prEN 50742 to IEC 62443-4-2 does not yield on its own. Where a device does not expose the artefact named by a requirement, the methodology records separately both the absence of the required function and the properties of any substitute artefact examined in its place.

Threats are identified with STRIDE, which derives threats from potential attacker goals as described in Section 2.2.3. The methodology then examines each security function from two directions. The first direction is the threat that the function mitigates. The second direction is the threat to which the function is itself exposed. A transmission checksum, for example, mitigates accidental corruption, yet it remains exposed to intentional forgery when the checksum is not cryptographic. This distinction prevents a function from being credited without an examination of its own weaknesses.

The final mapping step links each threat to the security function that addresses it, which makes the object of the test explicit. A test verifies whether the assigned security function is present and operates correctly for the affected asset, or it documents the gap where the function is absent or only partial.

3.3.3. Security Functions

While Section 2.2.3 identifies the threats and the security properties they violate, this section defines the countermeasure side. Fundamentally, security functions encompass the measures and techniques implemented in an IT or OT system to ensure the confidentiality, integrity, and availability of data and assets. They protect against unauthorised access, data loss, misuse, and other security-related threats. The security functions are grounded in the FRs of IEC 62443-3-3 and the corresponding CRs of IEC 62443-4-2 [27, p. 15] [8, p. 18], as well as in the technical measures required by prEN 50742 [7, p. 17].

- **Authentication:** Verification of the identity of a connecting user, system, or process to ensure that only authorised parties gain access.
- **Authorisation and Access Control:** Restriction of the permitted actions and accessible resources for an authenticated entity, typically managed through defined policies, role-based access control, and the principle of least privilege.
- **Confidentiality and Encryption:** Protection of data both in transit and at rest. This includes converting sensitive information into an unreadable format using cryptographic keys to prevent unauthorised information disclosure or alteration.
- **Integrity Protection:** Protection of software, firmware, and configuration data against unauthorised modification, enforced through mechanisms such as checksums, cryptographic signatures, or a verified boot process.
- **Security Logging, Auditing and Non-repudiation:** Continuous logging and monitoring of system activities to generate and retain evidence of security-relevant interventions. This ensures sufficient attribution and time reference to detect incidents, investigate misuse, and ensure non-repudiation.
- **Availability and Resilience:** Preservation of the intended system function under load, malformed input, or DoS conditions, combined with recovery mechanisms and a defined response to detected faults to prevent system or data loss.
- **Identification and Inventory:** Reliable identification of the component together with the enumeration of its constituent software, firmware, and configuration elements, for example through a software bill of materials, so that these assets can be tracked and evaluated against known vulnerabilities.

Not every gateway implements every function natively, and a function may be absent, partial, or delegated to an overarching organisational control. The methodology therefore examines, for each requirement, which of these functions the device actually provides, and derives a test that verifies the present function or documents the gap. Table 3.1 relates the STRIDE threat categories to these security functions and provides the bridge from the threat model to the tests.

Table 3.1.: Mapping of STRIDE threat categories to security functions (Based on [35] [47])

Threat Category	Security Function
Spoofing	Authentication
Tampering	Integrity Protection
Repudiation	Security Logging, Auditing and Non-repudiation
Information Disclosure	Confidentiality and Encryption
Denial of Service	Availability and Resilience
Elevation of Privilege	Authorisation and Access Control

3.3.4. Traceability Model

The methodology maintains a traceability chain (Based on [48, p. 36], [38, p. 82] [43, p. 21]). The chain links a derived component requirement to the evidence that supports its verification:

Requirement → Asset → Threat → Security Function → Test Objective → Test
Technique → Access Prerequisite → Evidence → Result

The nodes of the chain are defined as follows:

- **Requirement:** The derived component-level requirement (for example RQ-011) and its mapped IEC 62443-4-2 control (for example CR 2.1).
- **Asset:** The concrete gateway component to which the requirement applies.
- **Threat:** The attacker goal that targets the asset, derived via STRIDE.
- **Security Function:** The implemented countermeasure, together with any identified gap.

- **Test Objective:** The property to be demonstrated for the security function.
- **Test Technique:** The verification type, classified by the IEC 62443-4-1 SVV categories introduced in Section 2.3.5.
- **Access Prerequisite:** The network position required to run the test, expressed as a network access variant (Section 3.6).
- **Evidence:** The recorded artefact, such as a scan output, a packet capture, a log export, or a screenshot.
- **Result:** The observed outcome and its assessment against the acceptance criterion.

This chain ensures that every executed test serves a distinct requirement and that every requirement can be traced to concrete evidence.

3.4. Selection of Verification Techniques

The requirements define what must be verified. This section derives how the verification is performed.

3.4.1. State of the Art

Security testing of industrial components can draw upon several established frameworks and general testing paradigms. These approaches vary in their scope and focus, ranging from a general engineering testing paradigm through highly procedural IT security guidelines to abstract, compliance-oriented OT standards. They are evaluated against explicit criteria in Section 3.4.2.

Functional black-box testing verifies whether a specified behaviour is present without reference to the internal implementation, by comparing the actual output against the expected output for defined inputs [38, p. 82]. It is the general testing paradigm that requirements-based verification approaches, including the IEC 62443-4-1 SVV taxonomy discussed below, instantiate for the security domain. It is introduced here as an independent baseline because its central strength, direct requirement traceability, and its central

limitation, the absence of any assessment of resistance against deliberate misuse, are best established before the security-specific frameworks are discussed.

NIST Special Publication 800-115 provides a procedural, technical guide for network security testing and penetration testing. NIST SP 800-82 subsequently adapts these concepts for ICS. While they offer robust operational guidance for activities such as network discovery, port scanning, and vulnerability assessment, they remain inherently general and are not natively linked to the component-level obligations stipulated by the EU Machinery Regulation. Furthermore, NIST provides an overview of security testing techniques paired with applicable tools, for instance, mapping network discovery to Nmap [40, p. 65]. The core constituents of NIST SP 800-115 encompass review techniques (including documentation, log, and system configuration reviews), network sniffing, target identification and analysis techniques (such as network discovery and network port/service identification), alongside vulnerability scanning and penetration testing [40, p. iv].

IEC 62443 Security Verification (Parts 4-1 and 4-2) defines the normative state of the art for industrial cybersecurity. Part 4-1 provides a comprehensive taxonomy for SVV, encompassing requirements-based testing, threat mitigation, vulnerability analysis, and penetration testing [43, p. 37]. Although structurally complete and aligned with compliance mandates, the standard remains fundamentally abstract. It prescribes *what* must be verified but lacks concrete test cases or explicit tool specifications.

The Open Worldwide Application Security Project (OWASP) Web Security Testing Guide and Internet of Things (IoT) projects are established references for evaluating HTTP interfaces and Application Programming Interfaces (APIs). They provide actionable test scenarios for web services. However, they are primarily IT-focused and do not accommodate the deterministic OT fieldbus communication protocols (e.g., PROFINET, PROFIsafe) essential for this methodology. The OWASP IoT Security Testing Guide catalogues test cases across multiple hardware and software domains, including processing units, memory, firmware encompassing installed instances and update mechanisms, data exchange services, internal interfaces such as Inter Integrated Circuit (I²C) and Universal Asynchronous Receiver Transmitter (UART), as well as physical, wireless, and user interfaces [49].

MITRE ATT&CK for ICS serves as a structured knowledge base that catalogues real-world tactics (the overarching objectives of an adversary) and techniques (the specific methods employed to achieve these objectives) relevant to cyber threats in industrial environments (Techniques represent 'how' an adversary achieves a tactical goal by performing an action. For example, an adversary may dump credentials to achieve credential access.) [50]. It facilitates the structured derivation of test cases for functional security testing. Within

ATT&CK for ICS, tactics such as Initial Access and Lateral Movement precisely detail how adversaries attempt to compromise or bypass network gateways. Furthermore, MITRE enumerates concrete mitigation strategies for each identified attack technique, providing a pragmatic foundation for security assessments [51].

Fuzzing and robustness testing complements these frameworks by subjecting an interface to malformed, boundary, or high-volume input and observing whether the system maintains its specified behaviour [32, p. 29]. Unlike the four frameworks above, it is not tied to a single institutional standard but constitutes a general testing paradigm.

3.4.2. Evaluation of Approaches

The six approaches introduced in Section 3.4.1 are analysed against six criteria: traceability to requirements, availability of concrete test cases, reproducibility, suitability for an embedded OT target, compatibility with functional safety, and evidentiary value. Table 3.2 summarises their main strengths and limitations in the context of this thesis.

Table 3.2.: Comparison of established security testing approaches

Approach	Main strength	Main limitation
Functional black-box testing [38, p. 82]	Reproducibility and requirement traceability	Does not assess resistance against misuse
Abstract verification (IEC 62443) [43, p. 37]	Defines what must hold at component level	Provides no concrete, executable test cases
Penetration testing (NIST) [36, p. 296]	Realistic attack evidence	Tester-dependent and difficult to reproduce
Web and IoT testing (OWASP) [52]	Procedural depth for HTTP interfaces	Not tailored to OT fieldbus communication
Adversary-technique-based testing (MITRE ATT&CK for ICS) [50]	Structured, real-world catalogue of adversary tactics and techniques	Provides tactics and techniques, not executable test procedures
Fuzzing and robustness testing [32, p. 29]	Suited to error handling and availability	Requires careful monitoring and result interpretation

No single approach satisfies all criteria. Functional testing and abstract verification provide traceability but no attack perspective. Penetration testing, web-centric testing, and adversary-technique-based testing provide an attack perspective but weaker reproducibility, limited OT focus, or no executable test procedures. This gap between traceability-oriented and attack-oriented approaches motivates a combined strategy, which is developed after the

applicability of the existing company framework is assessed in Section 3.4.3 and justified in full in Section 3.4.4.

3.4.3. Applicability Assessment of the Company Framework

Ifm provides a manufacturer-specific security testing framework consisting of ten countermeasure-oriented instructions, denoted CM-001 to CM-010 [53]. Each instruction binds one countermeasure to one tool from Kali Linux, a Linux distribution that bundles pre-configured security testing tools, and to one acceptance rule. This framework is operational and organised around tools rather than derived requirements.

The framework cannot be adopted without adaptation. Several instructions assume interfaces that a general IT target exposes but the embedded SUT does not. CM-003 assumes an authentication endpoint, CM-005 assumes a Transport Layer Security (TLS) endpoint, and CM-006 assumes an operating-system shell. The SUT provides none of these. This observation motivates an explicit tool-to-target applicability assessment as a step of the methodology. For each planned tool, the methodology checks whether the target exposes the function that the tool assumes.

The applicability assessment produces one of four outcomes. A tool is *applicable* when the target exposes the assumed interface, so that the original procedure can be executed as written. A tool is *applicable with adaptation* when the target exposes a related but not identical interface, so that the original procedure does not hold as written, yet a modified procedure can still verify the intended property. This outcome covers both a tool whose scope must be narrowed to the interface actually present and a tool whose original procedure is inapplicable but which is redirected to an alternative endpoint that yields a usable observation. A tool is *not applicable* when the assumed interface is known in advance to be absent from the device or topology, which is an applicability finding and not a product failure. A tool is *blocked* when an environmental constraint prevents its operation.

3.4.4. Selected Verification Strategy

The selected methodology combines three elements derived from the state of the art. The first element is the verification taxonomy of IEC 62443-4-1 Practice SVV, introduced in Section 2.3.5, which distinguishes requirements-based testing, threat-mitigation testing,

vulnerability testing, and penetration testing [43, pp. 37]. The taxonomy anchors every test to a defined verification purpose. The second element is a set of requirement-driven test cases, derived from the fourteen requirements and their asset mapping, using testing procedures inspired by NIST and OWASP. The third element is a controlled reuse of the company tools, filtered by the applicability assessment.

These three elements correspond to three complementary layers of the derivation. Regulation (EU) 2023/1230 defines why a security property is required and yields the fourteen requirements. IEC 62443-4-2 defines what must hold at the component level through its CRs. IEC 62443-4-1 Practice SVV defines how each property is verified.

The combined approach meets the six criteria of Section 3.4.2. It preserves requirement traceability, and it yields concrete and reproducible test cases. It retains an attack perspective through the threat-mitigation and the vulnerability tests. It respects the safety priority by bounding the robustness tests to requirement-driven cases. It also remains compatible with the existing industrial validation process because it reuses the company tools where they apply and documents where they do not.

3.5. Test Case Development

This section defines how a requirement becomes a documented and repeatable test case. It describes the development process, the specification format, and the categorisation used to report the results.

3.5.1. Test Development Process

Each test case is derived in three steps. The first step is a critical review of the requirement. The implemented security functions and the identified gaps are cross-checked against the device documentation, so that a test verifies a real mechanism rather than an assumed one. The second step is test planning. For each requirement, one or more scenarios are defined. Each scenario records the property it proves, its technical justification, and its SVV category. The justification refers to established testing guidance, for example NIST SP 800-115 for network-level tests, MITRE ATT&CK for ICS for adversary techniques, and the OWASP Web Security Testing Guide for the HTTP interface [40, p. 1-1] [50] [52]. The third step produces the documented test case in a two-level format.

3.5.2. Test Specification

Every test case is specified on two levels. This separates the product-neutral definition from the environment-specific execution and supports reuse across device variants [54, p. 37].

The first level is the abstract test scenario. It is independent of the concrete laboratory and contains the following fields:

- **Traceability:** the requirement, its mapped CR, and the target asset.
- **Target Threat:** the STRIDE threat that the test exercises. A functional baseline that carries no adversarial goal is marked as not applicable.
- **SVV Category:** the verification type according to IEC 62443-4-1.
- **Objective:** the property to be demonstrated.
- **Steps:** the abstract procedure.
- **Acceptance Criterion:** the exact expected behaviour that constitutes a pass.

STRIDE applies at the level of the security requirement rather than to every individual test. A functional baseline or a pure availability demonstration carries no adversarial goal. For such a test case, the target-threat field is set to not applicable and records a functional or availability verification instead of a STRIDE category. A self-inflicted DoS test, in which the tester deliberately exhausts a resource of the device, is the one borderline case and is labelled accordingly.

Every availability or robustness test declares its acceptance criterion against two distinct references, because such a requirement is verified from two different perspectives. The first is a specification-conformance criterion bounded by the device's specified operating envelope, such as the maximum number of concurrent connections permitted by the product specification; it verifies whether the intended function is preserved within the declared range and is the criterion applied to the availability aspect of RQ-007. Observations made outside that envelope do not, by themselves, support a Pass or Fail verdict against this first criterion and are recorded as overload observations. The second is an adversarial criterion applied to requirements that mandate resistance to a DoS condition, such as

RQ-010 and its underlying CR 7.1, under which the component must maintain its essential functions when an attacker deliberately exhausts a resource. An adversary is not bound by the declared operating envelope, so a demonstrated and reproducible denial of an essential function is scored against this second criterion even when the triggering load lies outside the specified range. Here the declared envelope is treated as the potential weakness under evaluation, not as the acceptance threshold. The two criteria are reported separately so that a specification-conforming result and an adversarial DoS finding cannot be conflated.

The second level is the concrete execution. It binds the abstract scenario to the laboratory. It records the required network access variant, the applied tools, and the concrete commands, and it provides the fields for the result and the supporting evidence. The procedural depth of the concrete steps follows NIST SP 800-115 for network-level tests, the OWASP Web Security Testing Guide for the HTTP interface, and MITRE ATT&CK for ICS for adversary techniques in the OT domain [40, p. 1-1] [52] [50].

3.5.3. Test Categorisation

Each test case is classified along four dimensions. The first dimension is the security category. It groups tests by the security property under examination and provides the reader-facing structure of the results in Chapter 4. The methodology distinguishes seven security categories: identification and asset discovery, communication security, logging and traceability, availability and resilience, robustness, authentication and access control, and firmware and configuration integrity. Each category groups the tests that verify one or more of the security functions defined in Section 3.3.3, and thereby one of the IEC 62443-3-3 FRs that these functions are grounded in, as Table 3.3 shows.

The authentication and access control category combines the authentication and the authorisation function because both are examined jointly whenever an interface restricts access. Authentication establishes who is connecting, and authorisation establishes what the connecting entity is permitted to do. This combination also reflects IEC 62443-3-3 itself, where FR 1 is defined as Identification and Authentication Control.

The second dimension is the SVV category, which states the verification purpose. The third dimension is the network access variant, which states the required position on the communication path (Section 3.6). The fourth dimension is the execution and evidence status, which states what was achieved for a given test case.

A categorisation by FR was considered but rejected. As Table 3.3 shows, the mapping between security categories and FRs is many-to-many rather than one-to-one: “Authentication and access control”, for example, spans FR 1 and FR 2, while “Robustness” spans FR 3 and FR 7. Reporting strictly by FR would therefore fragment coherent test activities across multiple, only partially related result sections. The security-category scheme instead groups tests by the function they exercise in practice, which matches how tests are designed and executed, while traceability to the IEC 62443-3-3 FRs is preserved through Table 3.3.

Table 3.3.: Mapping of security categories to IEC 62443-3-3 requirements (Own compilation)

Security category	Security function (Sec. 3.3.3)	FR
Identification and asset discovery	Identification and Inventory	FR 1
Communication security	Confidentiality and Encryption	FR 3, FR 4
Logging and traceability	Security Logging, Auditing and Non-repudiation	FR 2, FR 6
Availability and resilience	Availability and Resilience	FR 7
Robustness	Availability and Resilience	FR 3, FR 7
Authentication and access control	Authentication, Authorisation and Access Control	FR 1, FR 2
Firmware and configuration integrity	Integrity Protection	FR 3

Table 3.4 maps each security category to the requirements it primarily addresses. The mapping is not exclusive because one test activity can support several requirements, and one requirement can appear in more than one category.

Table 3.4.: Mapping of security categories to regulatory requirements

Security category	Primary requirements
Identification and asset discovery	RQ-001, RQ-004, RQ-006, RQ-007
Communication security	RQ-001, RQ-002, RQ-014
Logging and traceability	RQ-003, RQ-008, RQ-009, RQ-012, RQ-013
Availability and resilience	RQ-007, RQ-010
Robustness	RQ-010
Authentication and access control	RQ-009, RQ-011, RQ-014
Firmware and configuration integrity	RQ-002, RQ-005

3.6. Security Test Environment

Security testing can disturb an industrial network. Active scanning or malicious input can interrupt communication and, in the worst case, affect a safety function. In OT, availability and safety hold the highest priority. Security tests must therefore never run in a production environment. A dedicated and isolated test environment is mandatory, in line with NIST SP 800-82 [46, p. 2].

The feasibility of a test depends on the position of the test system on the communication path. To structure the testing methodology for this work, three specific network access variants were defined. Each variant provides a different level of logical and physical access: direct access (Section 3.6.1), passive observation (Section 3.6.2), and active in-path manipulation (Section 3.6.3). Figures 3.1, 3.2, and 3.3 illustrate the three variants.

3.6.1. NV1: Direct Access

The test system communicates as a regular participant with the SUT and the controller over the switch. Network Variant 1 (NV1) supports all IP-based tests, including the IoT-Core HTTP services, and any test in which the test system acts as an unexpected communication partner [40, pp. 4-1, pp. 5-5] [45, p. 116].

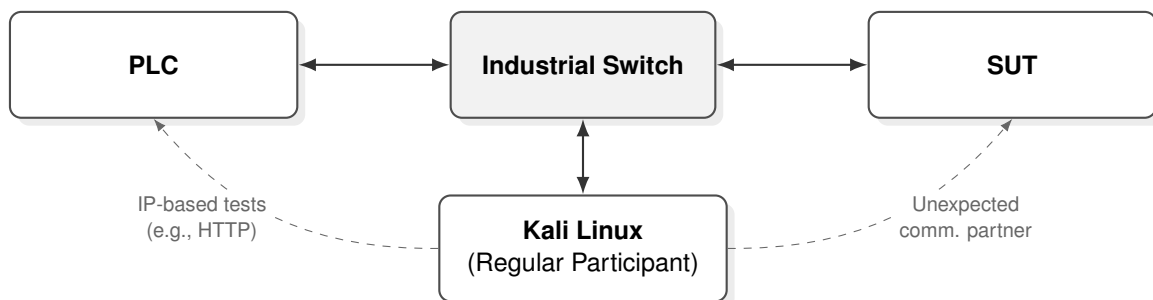


Figure 3.1.: Network Variant 1, direct access (Own illustration)

3.6.2. NV2: Passive Observation

A switch mirror or a network tap makes traffic visible that is not addressed to the test system, such as the cyclic PROFINET and PROFIsafe communication between the controller and the SUT. Network Variant 2 (NV2) requires a switch that supports and is configured for port mirroring. A network tap is an alternative to a mirror port. It duplicates

the traffic on a dedicated hardware path and avoids the frame loss that a mirror port can introduce under high load [40, p. 3-4] [45, p. 72, p. 116]. Address resolution spoofing is not a substitute because the cyclic traffic uses Ethernet-level addressing (EtherType 0x8892 and Discovery and Configuration Protocol (DCP)) rather than IP and Address Resolution Protocol (ARP) [55, p. 34].

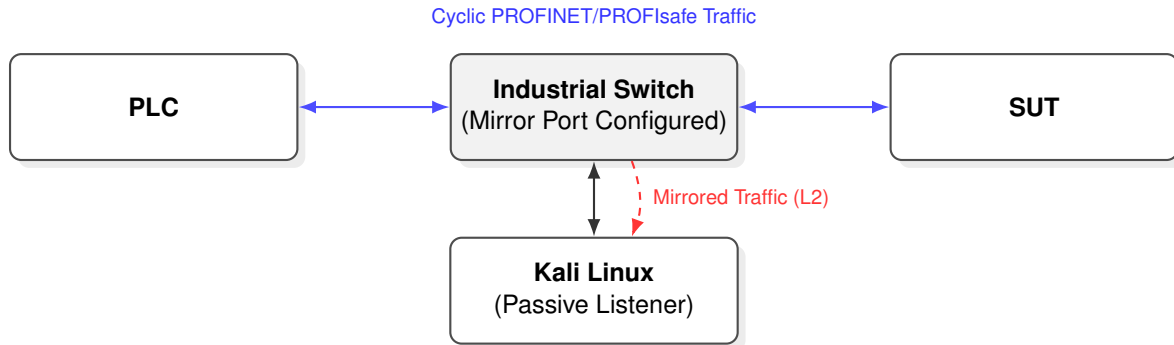


Figure 3.2.: Network Variant 2, passive observation (Own illustration)

3.6.3. NV3: Active In-Path Manipulation

The test system is inserted inline between the SUT and the switch and operates as a transparent bridge. Network Variant 3 (NV3) enables the controlled suppression, delay, or replacement of individual frames in transit [45, pp. 42, p. 45].

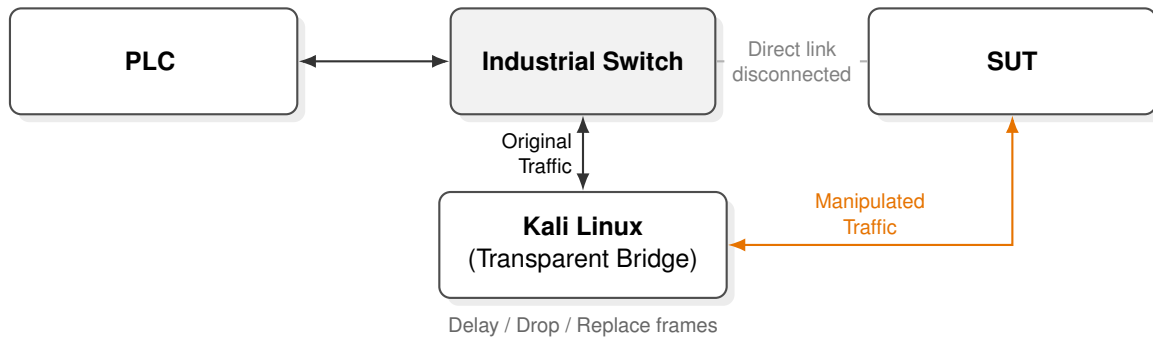


Figure 3.3.: Network Variant 3, active in-path manipulation (Own illustration)

3.6.4. Test System and Tooling

Security testing exercises an object under defined conditions and compares the actual behaviour with the expected behaviour [40, p. ES-1]. The SUT is evaluated as a compiled product without source access, only requirement documentation. The methodology

therefore uses dynamic black-box testing, also known as DAST. Static source analysis is out of scope, as stated in Section 3.2.3.

The methodology adopts Kali Linux as the standardised operating environment. Several options are considered. A general-purpose Linux distribution with individually installed tools offers flexibility, but it increases setup effort and reduces reproducibility. Commercial security suites offer integrated reporting, but their closed nature limits transparency and independent reproduction. Comparable open security distributions, such as Parrot OS, provide a similar toolset [56]. Kali Linux is selected because it bundles a maintained and pre-configured set of open-source auditing tools, which improves reproducibility and removes the need to develop custom exploit code [57]. Its use of the same platform as the company framework is a secondary benefit that supports transfer into the industrial process, not the primary reason for the selection. The tools are grouped by function: network discovery and service enumeration, HTTP and API interception, credential testing, and load or robustness testing. A tool is used only after the applicability assessment of Section 3.4.3 confirms that the target exposes the assumed interface.

The engineering tool for the safety configuration, Totally Integrated Automation (TIA) Portal, is intended only as a supporting instrument, for example to read identification and maintenance data or to observe the safety qualifier. It is not treated as a security testing tool.

3.7. Verification and Assessment Model

This section defines how a test outcome becomes a requirement verdict. The model deliberately avoids a numeric score because the practical phase executes only a subset of the specified tests, so a numeric score would suggest a completeness that the bounded demonstration does not have. The assessment is therefore qualitative and evidence-based [58, p. 144, 217].

3.7.1. Test Verdicts

The methodology distinguishes two separate dimensions: the execution status of a test and the verdict assigned to the corresponding requirement. The first answers whether the test was actually run and whether the evidence exists. The second answers whether the

requirement is satisfied on the evidence available. This separation is mandatory because the two dimensions operate on different objects: execution status is recorded per test case, while the requirement verdict is aggregated across all test cases associated with a requirement (Section 3.7.2). Execution status records what happened to the test activity. It distinguishes the following states [59, p. 62]:

- **Executed:** The test was run and the stored evidence supports a technical interpretation.
- **Not Applicable:** The assumed interface, function, or topology is absent. This is a technical applicability finding, not a product failure.
- **Blocked:** The test could not be run because of an environmental or tool constraint [59, p. 11].
- **Unevidenced:** The test was run, but no evidence artefact was retained, so the run cannot support a technical interpretation.

Requirement verdict records the assessment of the obligation itself. It distinguishes:

- **Pass:** The test case was executed, all mandatory acceptance criteria are satisfied, and the evidence is sufficient to demonstrate the security function.
- **Fail:** The test case was executed and the device demonstrably deviates from the obligation, or a mandatory security function was bypassed [59, p. 10].
- **Not Assessed:** No scored evidence exists for the requirement, or the condition is not yet testable in the executed scope.

A requirement can accordingly be assessed as Not Assessed even though none of its associated tests is itself unresolved, for example, if every test associated with a requirement is Not Applicable, Blocked, or Unevidenced, no individual test carries an ambiguous label, yet the requirement as a whole lacks any evidence to support a Pass or a Fail. The execution status and the requirement verdict therefore remain conceptually distinct. The status explains whether a given test produced usable evidence, while the verdict explains whether the requirement is satisfied on the evidence available across all its associated tests.

One rule governs how the absence of a function is scored, because absence can arise for

two distinct reasons that must not be conflated. A tool that is Not Applicable is an artefact of the test instrument, the interface that this specific tool assumes is absent, so the tool yields no verdict, neither Pass nor Fail, and is merely removed from the evidence set of the requirement. A capability that the device demonstrably lacks is an artefact of the device, and it is scored Fail against the derived requirement whenever the executed evidence positively demonstrates that the mandated capability is absent or deviates from the obligation. The decisive criterion is therefore not whether a function was found, but whether its absence was positively demonstrated by the evidence. Observing that one possible realisation of a function is absent is not, on its own, a demonstration that the requirement's mandated capability is absent, unless no alternative realisation remains and the acceptance condition is thereby unmet. Consequently, a Not Applicable tool never lowers a requirement to Fail. The requirement is assessed on the remaining evidence and defaults to Not Assessed where that evidence neither demonstrates the capability nor demonstrates its absence.

3.7.2. Requirement Assessment

The verdicts are aggregated per requirement according to a conservative rule that does not credit partial evidence. A requirement is assessed as Fail if any associated test case is Executed and the device demonstrably deviates from the obligation, or if a mandatory security function is bypassed. A Fail therefore takes precedence over any other outcome. A requirement is assessed as Pass only if every test case associated with it and applicable to it is Executed and passes with sufficient evidence, so that the requirement is fully covered by the executed scenario set. In every other case, including a requirement for which some but not all associated tests are Executed and passing while the remainder are Blocked or Unevidenced, the requirement is reported as Not Assessed, together with the reason. This conservative default means that incomplete test coverage is never reported as a Pass, even when no deviation was found.

3.7.3. Evidence Model

The assessment rests on one evidence rule that is applied without exception. A test is considered Executed only if a corresponding stored artefact exists. An artefact is a recorded output of the test, for example a tool log, a packet capture, a log export, or a screenshot. A reported test run without such an artefact does not qualify as Executed. It is recorded as Unevidenced and cannot, by itself, support a Pass at the requirement level. A tool

execution is not, by itself, a requirement verification. The observation it produces must be interpreted against the acceptance criterion, and this interpretation is kept separate from the raw observation. Expected behaviour taken from a test specification is never presented as an observed result.

The same rule governs the relationship to existing functional and safety tests. The SUT is a safety device, so some security-relevant properties are already exercised by functional or safety validation. Such a test does not, on its own, prove cybersecurity compliance because safety validation examines behaviour under random faults, whereas a security verdict requires an adversarial perspective and a security-specific acceptance criterion. Safety evidence is therefore reused only when it is re-evaluated against such a criterion. The organisational embedding of these verification activities into a secure development lifecycle, including the governance responsibilities, is described in Appendix C.

4. Implementation, Results and Discussion

This chapter applies the methodology developed in Chapter 3 to a physical SUT. It documents the implemented environment, the instantiation of the method, the collected evidence, the verification of the derived requirements, and the findings.

The practical work covers the direct-access variant NV1 defined in Section 3.6.1. The passive-observation variant NV2 and the in-path variant NV3 are part of the methodology but were not realised in the laboratory. The validation therefore covers the core NV1 process and those requirements whose acceptance conditions are observable from the direct-access position. The tests that depend on NV2 or NV3 remain specified but not executed.

4.1. Scope of the Practical Demonstration

This section defines the object, the environment, and the boundaries of the practical demonstration.

4.1.1. System Under Test

The SUT is the SRIO, a fieldbus-based safety component that exchanges safety-related input and output data with a higher-level safety controller over PROFINET and PROFIsafe. The evaluated unit identified itself over the IoT-Core interface as product `ifm-AL400S`, hardware version 1.4.0.2, hardware revision AA, and software revision 1.1.4.888. All reported results refer exclusively to this configuration.

Following the asset classification of Section 2.2.2, the SRIO is decomposed into hardware, software, and data assets. For the practical assessment, the externally accessible attack surface was divided into the HTTP-based IoT-Core diagnostic interface, the PROFINET and PROFIsafe communication path, the DCP configuration functions, and the firmware update mechanism. The first two form, respectively, the IT-side diagnostic path and the OT-side fieldbus path. The DCP and firmware functions are configuration-time extensions of the same non-safety path. Only the interfaces accessible from the implemented NV1 position were exercised. Internal debug and inter-processor interfaces, such as the System Communication (SysCom) link between the Communication Module (COM) and Safety CPU (SCPU) modules and the Joint Test Action Group (JTAG) interface, are included in the asset model as hardware assets but were not accessible within the practical test scope. The JTAG pads are sealed by the overmoulded housing of the SRIO, and the SysCom link is an internal inter-processor bus with no externally exposed connector. Their absence from the practical scope is therefore a physical-access constraint.

Table 4.1 summarises the assets and their protection objectives, using the hardware, software, and data categories of Section 2.2.2. It is not exhaustive, but it covers the main elements that are relevant for the security assessment.

Table 4.1.: Overview of assets and protection objectives

ID	Asset	Description	Protection objectives
A	Trusted Safety Function	SRIO correctly executes its specified safety function.	Integrity, Availability
B	Integrity of Safety Configuration	Safe behaviour is determined by the intended configuration.	Accountability, Authorisation, Integrity, Authenticity
C	Integrity and Authenticity of Safety-Relevant Process Data	Input, output and PROFIsafe data correspond to the actual safety state.	Integrity, Authenticity, Availability
D	Authenticity and Integrity of Safety Software	Bootloader and firmware remain authentic and unmodified.	Accountability, Authenticity, Integrity
E	Integrity of Safety Monitoring	Self-tests, plausibility checks, diagnostics and fault responses remain trustworthy.	Integrity, Availability

Continued on next page

Overview of assets and protection objectives (continued)

ID	Asset	Description	Protection objectives
F	Separation of Safety and Non-Safety Domain	A compromised COM system must not affect the integrity of the safety function.	Confidentiality, Integrity, Availability
G	Integrity of Operating Mode	Test and update functions must not be activated or used without authorisation.	Accountability, Authorisation, Integrity, Authenticity, Availability
H	SRIO Functionality	Availability of SRIO functionality shall be ensured.	Availability
I	Integrity, Availability and Confidentiality of Audit/Tracing Data	Evidence of interventions, configuration changes and installed software versions must be generated, retained for the mandated period and access-restricted.	Accountability, Availability, Confidentiality

4.1.2. Implemented Test Environment

The test environment used one switch and one flat address range. Figure 4.1 shows the physical topology, and Table 4.2 lists the components and their roles.

Table 4.2.: Components of the implemented test environment

Component	Address	Port	Role
Industrial switch	192.168.0.91	-	Layer-2 connectivity for all nodes
PLC	192.168.0.1	1	Safety controller and PROFINET/PROFIsafe counterpart
SRIO (SUT)	192.168.0.2	2	Device under test
Kali Linux test system	192.168.0.80	3	Security test system, connected through a USB 3.0 to Gigabit Ethernet adapter
Company laptop	192.168.0.7	4	Engineering station providing TIA Portal for planned identification and maintenance read operations

All nodes shared the 192.168.0.0/24 segment and one Ethernet broadcast domain. No Virtual Local Area Network (VLAN) separation was configured. The test system reached

the SUT and the controller directly over IP. The test object was the SRIO on port 2. The test initiator was the Kali system on port 3. The company laptop on port 4 served as the engineering station and provided TIA Portal for planned identification and maintenance read operations. These read operations were not part of the evidence-backed execution subset reported in this chapter. The available observation points were the direct request and response traffic of the test system and the HTTP responses of the IoT-Core service. Traffic that was not addressed to the test system was not necessarily visible because a switch forwards unicast frames only to the destination port. NV1 was therefore the realised position. NV2 and NV3 were not implemented.

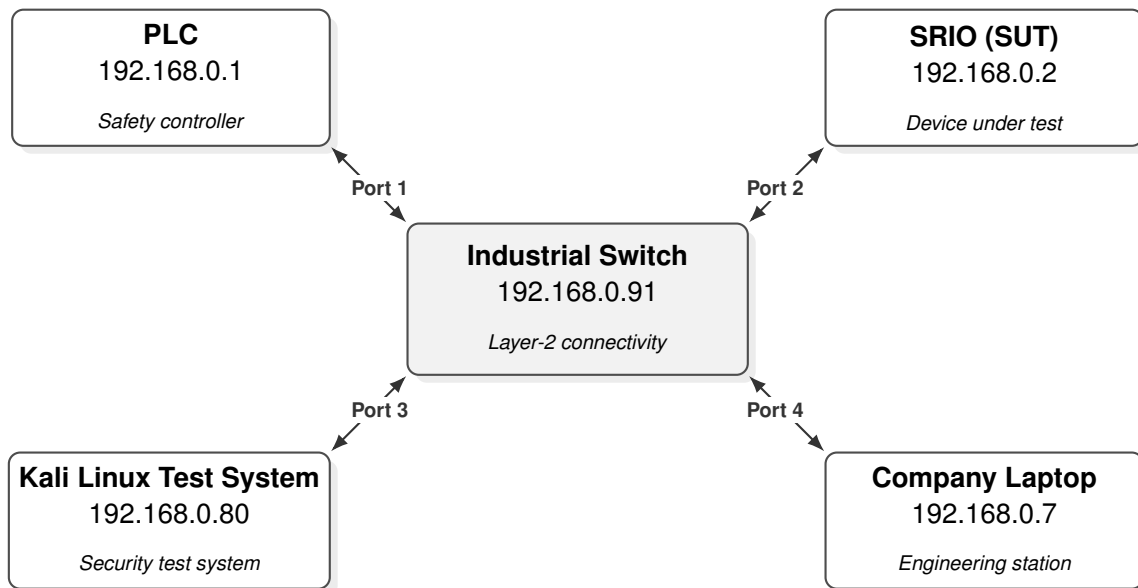


Figure 4.1.: Logical topology and addressing of the implemented test environment (Own illustration)

The device documentation defines organisational preconditions for secure operation. The intended use assumes the device in a protected zone behind a firewall. A firmware update is expected to be performed by a trusted person behind that firewall. A security risk assessment is required. The debug interface is primarily a physical interface. These preconditions bound the threat model of the practical tests and are relevant to the compensating-countermeasure allocation. Zone protection, trusted update procedures, and the system-level security risk assessment may address obligations that the component does not implement itself. They therefore inform the interpretation of component-capability findings in Section 4.4.

4.1.3. Practical Coverage and Limitations

The demonstration applied the methodology to the SRIO under the NV1 network position defined in Section 3.6.1. The test system acted as a direct IP participant on the shared switch. The passive-observation variant NV2 and the in-path variant NV3 were not realised. No mirror or Switched Port Analyzer (SPAN) port was configured because the available laboratory switch does not support port mirroring, and the test system was never placed inline, because rebuilding the setup for an inline bridge exceeded the available time. The cyclic PROFINET and PROFI-safe communication between the controller and the SUT was therefore not observed, and no claim is made about that traffic.

The empirical coverage was bounded by the single test object, the implemented topology, the available interfaces, and the NV1-compatible activities. Within NV1, the evaluation focused on the IoT-Core HTTP interface and on tests in which the test system acted as an unauthorised communication partner. Where a company tool assumed an interface that the device does not expose, the test was recorded as an applicability finding rather than as a failure, following Section 3.4.3. The scope of this practical validation must be distinguished from the achievement of the thesis objective, which is to develop and validate a systematic methodology for verifying security functions rather than to execute every technically possible test.

4.2. Instantiation of the Methodology

This section instantiates the abstract methodology of Chapter 3 on the SRIO.

4.2.1. Requirement, Asset and Interface Mapping

This section instantiates, for the SRIO, the abstract asset-and-threat mapping step defined in Section 3.3.2. Each of the fourteen derived requirements (Table B.1) is bound to the concrete asset it protects, the STRIDE threat it defends against, and the security function that provides the defence. These three columns follow the order of the traceability chain defined in Section 3.3.4. Table 4.3 records the resulting mapping. It summarises, at requirement level, the working test-case catalogue introduced in Section 3.5.2. The individual abstract scenarios that this summary aggregates, including the concrete interface,

the network access variant, and the evidence for each, are reported by security category in Section 4.3, and the executed subset is assessed in Section 4.4.

The columns are read as follows:

- **Asset:** the entries of Table 4.1 affected by the requirement. A requirement typically protects more than one asset.
- **Threat:** the STRIDE categories exercised across the requirement's test scenarios. A requirement is often examined from more than one threat angle. For example, RQ-001 is tested against an impersonated device (Spoofing), a manipulated frame (Tampering), and a flooding condition (Denial of Service). Following Section 3.5.2, a scenario with no adversarial goal, such as a pure functional or identification baseline, does not contribute a threat entry.
- **Security function:** for each threat listed, the function of Section 3.3.3 that addresses it, following the STRIDE-to-function mapping of Table 3.1 (for example, Spoofing pairs with Authentication, Information Disclosure with Confidentiality and Encryption). Where a requirement concerns discovery rather than a protective mechanism, this is instead the identification-and-inventory function that Table 3.3 also grounds in Section 3.3.3.

The required network access variant (Section 3.6) is not a column of Table 4.3. Access is not a property of the requirement, the asset, or the threat, but rather of the individual test technique chosen to verify a security function. It is the last element of the traceability chain to be fixed, following Test Objective and Test Technique in Section 3.3.4. Different scenarios for the same requirement can therefore require different access variants, so the position is decided case by case for each concrete test, and is reported alongside that test in Section 4.3.

The underlying derivation chain from the Regulation through prEN 50742 and IEC 62443-4-2 is documented in Appendix B.

Table 4.3.: Instantiated requirement-to-asset mapping for the SRIO

Req.	Asset	Threat	Security function
RQ-001	A, C, E, F	Spoofing, Tampering, Denial of Service	Authentication, Integrity Protection, Availability and Resilience

Continued on next page

Instantiated requirement-to-asset mapping for the SRIO (continued)			
Req.	Asset	Threat	Security function
RQ-002	C, G	Tampering	Integrity Protection
RQ-003	G, I	Repudiation	Security Logging, Auditing and Non-repudiation
RQ-004	B, D	Tampering, Information Disclosure	Identification and Inventory; Confidentiality and Encryption
RQ-005	B, D, F	Tampering	Integrity Protection
RQ-006	D	Information Disclosure	Identification and Inventory; Confidentiality and Encryption
RQ-007	D, F, H	Denial of Service	Availability and Resilience
RQ-008	D, F, G, I	Repudiation	Security Logging, Auditing and Non-repudiation
RQ-009	B, D, F, I	Repudiation	Security Logging, Auditing and Non-repudiation
RQ-010	A, E, F, H	Tampering, Denial of Service, Elevation of Privilege	Integrity Protection, Availability and Resilience, Authorisation and Access Control
RQ-011	B, F, G	Spoofing, Tampering, Elevation of Privilege	Authentication, Integrity Protection, Authorisation and Access Control
RQ-012	F, I	Repudiation	Security Logging, Auditing and Non-repudiation
RQ-013	D, I	Repudiation	Security Logging, Auditing and Non-repudiation
RQ-014	H, I	Information Disclosure, Elevation of Privilege	Confidentiality and Encryption; Authorisation and Access Control

4.2.2. Applicability Assessment of the Company Framework for the SRIO

Table 4.4 records, for each of the ten company tests, whether the interface or precondition that the tool assumes is present on the SRIO in principle. This determination is made before execution and is a property of the device and the topology, not of a completed test run. Whether an applicable tool was actually executed, and what it found, is reported in Section 4.3. The CM-002 not-applicable outcome is a property of the flat, single-

switch laboratory topology rather than of the device itself. A production deployment following the zone-and-conduit segmentation of IEC 62443-3-3 would typically provide the second network zone that CM-002 assumes, so this specific applicability outcome does not generalise to a segmented deployment and is reported here as a laboratory-topology constraint rather than as a general property of the SRIO. CM-003 assumes an authentication endpoint, which the device does not expose. CM-006 assumes an operating-system shell, which the embedded device does not provide. CM-005 assumes a dedicated TLS endpoint on Transmission Control Protocol (TCP) port 443. The device’s diagnostic service is documented as plain HTTP on port 80 and provides no separate Hypertext Transfer Protocol Secure (HTTPS)/443 endpoint.

Table 4.4.: Applicability of the company framework tests to the SUT

CM	Tool	Applicability outcome
CM-001	Nmap	Applicable; the device is reachable over IP, so a network port scan can enumerate its interfaces
CM-002	Nmap	Not applicable to the topology; no zone or conduit model in a flat segment
CM-003	Hydra	Not applicable; no authentication endpoint is exposed
CM-004	OWASP ZAP	Applicable with adaptation; the device exposes a web interface, but only a single anonymous read-only role, so a multi-role comparison is not applicable
CM-005	SSLscan	Applicable with adaptation; the original procedure targets a dedicated HTTPS/443 endpoint, which the device does not provide, so the probe is redirected to the plain HTTP service to obtain a usable transport-security observation
CM-006	Lynis	Not applicable to the device; no operating-system shell
CM-007	Nikto	Applicable; the device exposes a web interface that a web-server scanner can probe
CM-008	OpenVAS/GVM	Blocked; the scanner could not be made operational in the laboratory setup
CM-009	Siege	Applicable; the device exposes an HTTP service that a load-generation tool can address
CM-010	boofuzz	Applicable; the device exposes an HTTP service that a fuzzing tool can address

4.2.3. Selected Test Cases

Table 4.5 follows the security-category structure of Table 3.4 and records, category by category, which method was selected to test it on the SRIO. The requirement column repeats the primary requirements of Table 3.4 exactly, so the two tables can be read side by side. The abstract mapping states what a category is intended to cover, and this table states what was actually selected under the NV1 position. A selected method can support

only part of a category's requirement list; where this occurs, the covered requirements are named explicitly. CM-002, CM-003, CM-006, and CM-008 are not selected as an executed method in any row because none is applicable to the SRIO or its topology (Table 4.4). This is a property of this specific device and topology, not of the company framework in general, since every countermeasure test is associated with at least one requirement category by design in the abstract framework of Section 3.4.3.

Table 4.5.: Test methods and regulation requirements per security category

Security category	Primary requirements	Selected method
Identification and asset discovery	RQ-001, RQ-004, RQ-006, RQ-007	Nmap (CM-001), Nikto (CM-007); IoT-Core inventory reads
Communication security	RQ-001, RQ-002, RQ-014	SSLscan
Logging and traceability	RQ-003, RQ-008, RQ-009, RQ-012, RQ-013	IoT-Core error log, systick
Availability and resilience	RQ-007, RQ-010	Siege (CM-009)
Robustness	RQ-010	boofuzz (CM-010)
Authentication and access control	RQ-009, RQ-011, RQ-014	OWASP ZAP, IoT-Core (CM-004); Hydra (CM-003) not applicable
Firmware and configuration integrity	RQ-002, RQ-005	none selected for NV1; requires the engineering tool or NV3

4.3. Test Execution and Results

The two-level test catalogue derived in Chapter 3 defines abstract test scenarios for all fourteen requirements, including the tests that depend on the passive-observation variant NV2 and the in-path variant NV3. Only a subset of these was executed in the practical phase, the applicable company tests and the requirement-oriented tests that the NV1 position supports. This section reports the seven security categories of Table 3.4 in the same order as Table 4.5, so that the abstract mapping, the selected method, and the executed result can be read together. The tests that require NV2 or NV3 are described as designed but not executed. Every reported result is limited to what the stored evidence shows. A tool execution is not treated as requirement verification, and an expected outcome from a test specification is not presented as an observed result.

Each category is introduced by the same reasoning: the machine-level obligation of Regulation (EU) 2023/1230 from which the addressed component requirements are derived, what must therefore be checked on the SRIO, how that property can be tested and which company countermeasure applies, and the general finding on the device, grounded in the stored evidence. Most performed tests are stored in some form as evidence, depending on the test as a text file, an Hypertext Markup Language (HTML) report, a log file, or a screenshot. Where an activity produced no stored artefact, this is stated explicitly and the observation is marked accordingly. The exact tool commands for the executed activities are listed in Appendix D.

4.3.1. Identification and Asset Discovery

Annex III, Section 1.1.9 requires that safety-critical software and data be identified (RQ-004, RQ-006), that this identification be available at all times in an easily accessible form (RQ-007), and that the connection of another device does not lead to a hazardous situation (RQ-001). This category therefore examines whether the SRIO exposes a discoverable, enumerable non-safety attack surface, and whether it can produce accurate hardware, software, and firmware identification data on request, at product granularity. Nmap (CM-001) and Nikto (CM-007) enumerate the reachable non-safety services, and a scripted query of the IoT-Core `deviceinfo` and `firmware` endpoints reads the identification data directly. Both run under NV1 (Table 4.5). The Software Bill of Materials (SBOM) completeness analysis and the data-signature uniqueness check belong to the abstract catalogue but were not executed. A complete SBOM is the practical expression of the component-inventory capability (CR 7.8 [8, pp. 68]) to which RQ-004, RQ-006, and RQ-007 are mapped (Appendix B.3). It would list every installed software and firmware component together with its version, rather than only the top-level identifiers that a product-granularity read exposes. The activities reported below show which services are reachable and confirm that safety-relevant software is identifiable at product granularity, but they do not establish a complete SBOM, and they do not support a hazardous-situation judgement.

Attack-Surface Enumeration This activity examined the externally reachable services on the non-safety path. It relates to CM-001 and CM-007, to requirement RQ-001, and to asset F, the separation of the safety and the non-safety domain. It targets the IoT-Core HTTP path and was run under NV1. The commands are listed in Appendix D.1.

A full TCP port scan and a top-ports User Datagram Protocol (UDP) scan were run

against the SUT with Nmap, each repeated on two separate dates to confirm reproducibility [60]. Both TCP runs (`-sS -sV -T3`) covered all 65535 ports and consistently reported one open port, port 80, which returned HTTP responses without a server banner. All other TCP ports were reported as closed. The open HTTP port is consistent with the IoT-Core diagnostic interface. Both UDP runs of the top 200 ports reported port 161 as open, identified as Simple Network Management Protocol (SNMP), and port 49152 as open or filtered. An open SNMP port is consistent with a PROFINET device because the PROFINET standard uses SNMP for network diagnostics [61, p. 11].

The externally reachable services on the non-safety path are therefore the HTTP service on port 80 and the SNMP service on port 161. On this device the physical debug interface is encapsulated and not reachable, so the scan characterises only the reachable network service surface, comprising the HTTP and SNMP services, and not the encapsulated physical debug path. The Nikto report lists missing HTTP security headers, including content-security-policy, x-content-type-options, strict-transport-security, and permissions-policy. The report also flags several `/actuator/*` paths as Spring Boot Actuator endpoints that return JavaScript Object Notation (JSON).

Nikto is a generic web-server vulnerability and misconfiguration scanner. It enumerates common web paths and matches responses against known vulnerability signatures [62]. Its findings here therefore characterise the IoT-Core diagnostic HTTP service, which is the device's non-safety web interface by design, and not the physical debug interface discussed in Section 4.1.1, which remains encapsulated and outside the practical scope.

The enumeration supports RQ-001 at the level of the non-safety path. It shows which services a connecting device can reach. It does not, on its own, demonstrate a hazardous situation. The SNMP service on port 161 was deliberately excluded from the practical scope because it was not examined beyond enumeration. Its authentication, information-disclosure and logging implications remain open items for a future test campaign.

Identification-Surface Inventory This activity enumerated the identification data that the device exposes. It relates to requirements RQ-004, RQ-006, and RQ-007, and to assets B and D. It targets the IoT-Core `deviceinfo` and `firmware` paths and was run under NV1. The commands are listed in Appendix D.2.

The identification interface was queried over HTTP. The interface required no credentials, so the read operation was anonymous. For each queried path the device returned a JSON response with status code 200 that contained the requested identifier. The read operations

covered the hardware version and revision, the software revision, the COM and SCPU firmware and bootloader versions, the host firmware version, and the fieldbus firmware version. A successful read operation is recognisable by the returned status code 200, which indicates a successful response. A separate single read of the software revision also returned status 200, which documents the baseline availability of the interface.

The device exposes top-level version identifiers for the COM firmware, the SCPU firmware, the bootloaders, and the fieldbus firmware, all through anonymous read access. The inventory supports RQ-004, RQ-006, and RQ-007 at product granularity. It confirms that safety-relevant software is identifiable and available on request. It does not establish a complete SBOM. The versions of third-party components, such as the communication stack or the real-time operating system, are not individually exposed. The visible data is therefore an identification surface rather than a complete installed-software inventory or a full SBOM.

4.3.2. Communication Security

Annex III, Section 1.1.9 requires that the connection of another device does not lead to a hazardous situation (RQ-001) and that hardware components transmitting signal or data relevant to connection or access to safety-critical software be protected against corruption (RQ-002). Section 1.2.1 requires that access to tracing-log data be restricted (RQ-014). This category therefore examines whether the safety-relevant communication path is protected from tampering, spoofing, or injection by an unexpected participant, and whether data carried on the reachable non-safety diagnostic path, including the tracing log, is exposed in a way that an unauthorised observer could exploit. RQ-014 is examined here only from this confidentiality-in-transit angle. Whether the interface itself enforces the access restriction to a reasoned conformity request is a separate, access-control angle examined under authentication and access control (Section 4.3.6). The safety-telegram integrity-rejection, in-path manipulation, and watchdog-passivation scenarios require the NV2 or NV3 position or the engineering tool and were designed but not executed. Under NV1, an SSLscan probe checks whether the IoT-Core HTTP service uses transport encryption. The original CM-005 procedure is not applicable to this device (Table 4.4), so the probe is run directly against the tracing-log path and reused from the identification-surface inventory read (Table 4.5). The transport observation reported below is therefore the only activity executed under NV1. It also supports RQ-004 as a secondary contribution, alongside its primary categorisation under identification and asset discovery (Section 4.3.1). Had a TLS endpoint been present and correctly configured, it would have

provided confidentiality and integrity for the transported identification and tracing-log data against a passive or active on-path observer, closing the defence-in-depth gap that the absence of TLS otherwise leaves open [63].

Cleartext Transmission of the Identification and Tracing-Log Data This activity examined the transport protection of the identification data and of the tracing-log data. It relates to CM-005, to requirements RQ-004 and RQ-014, and to assets D and I. It targets the IoT-Core HTTP service, including the `errorlog/loglist` path that carries the tracing-log content, and was run under NV1. The commands are listed in Appendix D.3.

SSLscan verifies the TLS configuration of a TLS endpoint. It probes for an SSL or TLS handshake on a given host and port and reports the enabled protocol versions and the certificate [64] [65]. The tool was directed at the HTTP service on port 80. The saved output shows that no SSL or TLS protocol version, from SSLv2 through TLSv1.3, is enabled, and that no certificate can be parsed. The output therefore confirms that port 80 does not provide a TLS endpoint, so the service is plain HTTP without transport encryption for every path served on it. The identification data of Section 4.3.1 observed in the test system’s own requests and responses was therefore transmitted in plaintext.

The same absence of transport encryption applies to the tracing-log path. A dedicated SSLscan run against the identical HTTP service, saved separately as evidence for RQ-014, confirms that the tracing log read at `errorlog/loglist` is served without a TLS option. A packet capture intended to show the tracing-log content itself in cleartext was planned but was not saved as a stored artefact, so the log-content-specific claim is reported as an unscored observation. The scored finding for RQ-014 is limited to the shared absence of a TLS endpoint. The absence of TLS is a device characteristic, not a failed TLS test. It is relevant to RQ-004 and to RQ-014’s confidentiality-in-transit angle as a defence-in-depth observation. A confidentiality impact requires two further conditions. The transmitted data must be sensitive, and an unauthorised party must be able to observe it. The second condition depends on the NV2 position, which was not realised. The confidentiality claim is therefore limited to the cleartext transmission that the test system observed in its own requests and responses.

4.3.3. Logging and Traceability

Annex III, Section 1.1.9 requires evidence of any intervention in hardware components relevant for connection or access to safety-critical software (RQ-003), evidence of any intervention in the installed software (RQ-008), and evidence of any modification of the installed software or its configuration (RQ-009). Section 1.2.1 requires a tracing log of intervention data retained for five years (RQ-012). This category therefore examines whether the device generates, retains, and correctly time-references evidence of interventions, and whether that evidence survives a power cycle and remains attributable over the mandated retention period. This time-reference expectation derives from the auditable-events capability (CR 2.8/2.11 [8, p. 47, p. 49]) and its associated timestamp property, to which RQ-003, RQ-008, RQ-009, and RQ-012 are mapped (Appendix B.3). No company countermeasure targets logging directly. The selected method is a requirement-oriented scenario that reads the IoT-Core error log and systick counter before and after an induced event and a cold restart (Table 4.5). The software-installation and configuration-modification evidence scenarios for RQ-008 and RQ-009 require an induced software or configuration change. Only the baseline was read, so they were not fully executed. The intervention-evidence and time-attribution activity reported below is the only fully executed scenario in this category and yields the two Fail verdicts of Table 4.7.

Intervention Evidence and Time Attribution This activity examined what evidence the device records for an intervention and how it is time-referenced. It relates to requirements RQ-003, RQ-008, and RQ-012, and to assets I and G. It targets the IoT-Core `errorlog` and `systick` paths and was run under NV1. The commands are listed in Appendix D.4.

The machine-level requirement requires a tracing-log capability, and usable intervention evidence requires a time reference that remains unambiguous across the relevant operating states. No dedicated tracing log was exposed through the examined interface. The available substitute artefact is an error log with a relative system tick, and it is cleared at a cold restart. The RQ-003 verdict therefore rests on a capability finding rather than on the specific event induced. The deficiency is the structural absence of a dedicated tracing-log function and the demonstrated inadequacy of the only available substitute artefact, and both properties hold independently of which intervention is triggered. This is a different situation from RQ-008 and RQ-009, whose acceptance conditions turn on whether a particular software-installation or configuration event is recorded, and which therefore remain Not Assessed until such an event is induced. The induced input fault described below is used only to elicit and characterise the behaviour of the substitute artefact. It is not treated as a stand-in for an intervention in a connection-relevant hardware component.

The activity therefore records both the apparent absence of the required tracing-log function and the properties of the substitute artefact.

The error log and the system tick counter were read over HTTP in a controlled sequence. The sequence recorded a baseline, an induced input fault by short-circuiting pins 1 and 3 of a digital input of the SRIIO, a cold restart of the device by removing power, and a second induced input fault. Table 4.6 lists the recorded system tick values and the number of log entries at each step.

Table 4.6.: Recorded system tick values and error-log entry counts

Step	System tick	Log entries
Baseline	20969	0
After event 1 (input fault)	68629	2
After cold restart	20459	0
After event 2 (input fault)	44869	2

At the baseline the error log was empty. The short-circuit at the digital input triggered a fault, which produced two error entries on the safety Central Processing Unit (CPU). Each entry carried a relative system tick value but no absolute timestamp and no actor identity. After the cold restart the error log was empty again, and the system tick had dropped, as Table 4.6 shows. The second input fault produced two new entries with the same recorded error code. Because the system tick is reset by a restart, an entry recorded after the restart can carry a lower tick than an earlier entry, although it occurred later in real time.

The absence of a dedicated tracing log is the primary finding for the required function. The substitute error log does collect evidence of connection-loss events, but it carries only a relative system tick and no actor identity. The system tick resets at a cold restart, so entries cannot be ordered across a power cycle. An entry recorded after the restart can carry a lower tick than an earlier entry, which confirms the reset. The log content also did not persist across the cold restart. The two entries from the first event were absent afterwards. This evidence-based observation directly demonstrates cold-start deletion in the substitute artefact and supports the conclusion that the retention capability is not demonstrated for RQ-012.

4.3.4. Availability and Resilience

Annex III, Section 1.1.9 requires that identification information of safety-relevant installed software be available at all times (RQ-007). Section 1.2.1 requires that the control system withstand reasonably foreseeable malicious attempts that could lead to a hazardous situation (RQ-010). Both requirements concern the availability of the SRIO in general, including its safety function under a DoS condition. The practical scope here is bounded to the IoT-Core HTTP interface under NV1, so the activities reported below characterise only the non-safety diagnostic path and do not, by themselves, establish availability of the safety function itself. This category therefore examines whether the diagnostic HTTP service remains available under connection load and within its specified operating envelope (RQ-007), and whether the device resists a network or link-layer flood without the disturbance propagating to the safety function (RQ-010, domain-separation aspect). Siege (CM-009) load-tests the IoT-Core HTTP service at the framework default of 50 connections and near the device's specified two-connection envelope at 3 connections and a manual connection-hold procedure occupies both permitted slots and probes a third request. A network and link-layer flood (hping3, macof) was attempted but not captured as evidence [66] [67]. The domain-separation confirmation itself requires the NV2 position and was designed but not executed. The load runs reported below produce specification-conformance observations, and the connection-hold procedure produces a demonstrated DoS finding under the adversarial criterion, while the requirement-level verdict for RQ-010 remains Not Assessed pending the NV2 position. The layer-two flood produced no usable evidence.

Availability under Connection Load This activity examined the availability of the identification interface under connection load. It relates to CM-009, to requirements RQ-007 and RQ-010, and to assets H and F. It targets the IoT-Core HTTP service and was run under NV1. The commands are listed in Appendix D.5.

A load test with Siege was run against the IoT-Core HTTP service at 50 and at 3 concurrent connections. The two load levels reflect the framework default and a near-envelope stress case for the SRIO, whose requirement documentation permits at most two concurrent connections. The console output and the Siege summary logs are stored as evidence [68].

The acceptance criterion has two references here, following Section 3.5.2. Under the first, specification-conformance criterion, availability is assessed only for the number of concurrent connections that the product specification permits, so both the 3-connection and 50-connection runs lie outside the declared two-connection range and cannot, by

themselves, support a Pass or Fail verdict for the specified operating condition. Under the second, adversarial criterion for the DoS aspect of RQ-010, an attacker is not bound by that envelope; this criterion is evaluated through the connection-hold procedure below.

The stored summaries show that the service responded to a large number of requests with HTTP status 200 and that a substantial number of connections failed in the same runs. The 50-connection runs recorded 1,273 successful transactions and 407 failed connections in total. One run completed no transaction and failed all 50 connection attempts. The 3-connection run recorded 332 successful transactions and no failed connections. These results document behaviour under the tested loads, but the tested loads exceeded the specified maximum and therefore do not establish availability within the product's declared operating envelope under the specification-conformance criterion. The mixed pattern of successful and failed connections in these Siege runs is not, on its own, a clean denial of an essential function. The deliberate resource-exhaustion case that the adversarial criterion targets is evaluated separately through the connection-hold procedure below.

The connection-hold procedure was executed against the identification interface. The job listing shows two connection-holding processes running as intended. While both connections were held, a third request to the identification endpoint did not receive a response within the five-second timeout. After one held connection was released, the same request again returned HTTP status 200. The evidence therefore demonstrates that occupying the device's entire two-connection budget denies the diagnostic HTTP service to a further client, and that the endpoint recovers once a slot is released. Under the adversarial criterion of Section 3.5.2, this is a demonstrated and trivially reproducible DoS against a safety-related component: because an attacker is not bound by the declared two-connection envelope, the small connection budget is the weakness under evaluation rather than the acceptance threshold, and CR 7.1 requires the component to maintain its essential functions under such a condition. The finding is conclusive for the IP-reachable diagnostic interface. The requirement-level verdict for RQ-010 nonetheless remains Not Assessed, because the domain-separation aspect, whether this denial propagates to or spares the cyclic safety function, can only be observed from the NV2 position, which was not realised, and because the activity exercised a single endpoint.

Under the specification-conformance criterion, the load runs do not support a Pass or Fail verdict for the declared operating envelope. Under the adversarial criterion, the connection-hold procedure demonstrates a denial of the diagnostic interface, but a statement about domain separation or about the safety function itself is not possible under NV1 because the safety channel was not observed, which is why RQ-010 remains Not Assessed overall.

Layer-Two and Network Flooding This activity examined the resilience of the device against a link-layer and network flood. It relates to requirement RQ-010 and to assets H and F. It targets the shared Ethernet segment and was run under NV1. The commands are listed in Appendix D.6.

The planned packet and Media Access Control (MAC)-address flooding activity is not counted as an evidence-backed execution. No complete terminal capture, packet capture, or independent record of the reported device indication is available. The activity is therefore recorded as Unevidenced for assessment purposes and does not support a requirement verdict. To observe the effect of such a disturbance on the cyclic safety communication, the passive-observation variant NV2 would be required.

4.3.5. Robustness

Section 1.2.1 requires that the control system withstand reasonably foreseeable malicious attempts that could lead to a hazardous situation (RQ-010), examined here specifically under malformed input rather than under volume load. This category therefore examines whether the HTTP service maintains its specified behaviour when it receives malformed requests, and whether any resulting disruption remains confined to the non-safety interface. boofuzz (CM-010) fuzzes the request line of an HTTP GET request against the IoT-Core diagnostic service under NV1 (Table 4.5) [69]. The fieldbus-discovery and safety-frame fuzzing scenarios require the NV2 or NV3 position and were designed but not executed. The activity reported below shows a temporary service disruption and an unstored fault-indication observation; neither, by itself, supports a Pass or Fail verdict for RQ-010.

Robustness under Malformed Input This activity examined the robustness of the HTTP service against malformed input. It relates to CM-010, to requirement RQ-010, and to asset H. It targets the IoT-Core HTTP service and was run under NV1. The commands are listed in Appendix D.7.

A fuzzing test was run with boofuzz against the HTTP service on port 80. The saved script fuzzes the request line of an HTTP GET request. The boofuzz session databases are stored as evidence. The session record shows successful processing of the initial test cases, followed by a failed reconnection after a subsequent malformed request. The HTTP service became reachable again approximately 41 seconds later. The triggering input and the internal cause of the interruption were not established.

During the run, the device visibly indicated a fault condition on its status indicators. The indication changed as the run continued and had not fully cleared after the test was aborted. A manual read of the error log taken after the run showed one additional **FATAL**-level entry and three additional **ERROR**-level entries that were not present in the baseline of Section 4.3.3. This reading was not captured to a stored file and is therefore reported as an observation rather than as evidence. Independently of the specific error codes involved, which are not interpreted here, the indicator and error-log behaviour together show that the device entered a fault state as a consequence of the HTTP-level fuzzing. This is not expected behaviour and warrants closer review by the development department. Because the observation concerns safety monitoring (asset E), it is also relevant to the domain-separation obligation of RQ-001 (Section 4.3.1, asset F). It suggests, without yet proving, that the disruption is not confined to the non-safety interface.

The stored evidence therefore shows a temporary disruption of the HTTP service on the non-safety path under malformed input, followed by recovery, together with an unstored observation of a device fault indication. The disruption requires follow-up by development. Its causation, reproducibility, and effect on the safety function are not established because the safety channel was not monitored and the fault indication was not captured as a stored artefact. The activity therefore supports an observation but not a complete requirement verdict for RQ-010. The requirement remains Not Assessed under the conservative aggregation rule. An unresolved device fault state of this kind falls within the scope of IEC 62443-4-1's Defect Management practice. Under the governance model of Appendix C, it would be escalated to Software/Firmware Development in a consulted role and tracked by Security Engineering. Whether this specific finding has been formally raised through that process is outside the scope of this thesis and is noted here as an open item for the responsible team.

4.3.6. Authentication and Access Control

Section 1.2.1 requires that modifications to safety-relevant settings or rules that could lead to a hazardous situation be prevented (RQ-011) and that access to tracing-log data be restricted to a reasoned conformity request (RQ-014). Section 1.1.9's requirement to collect evidence of modification of the installed software or configuration (RQ-009) is also examined here because the same write attempts that probe access control also serve as its modification-evidence baseline. This category therefore examines whether the device authenticates or authorises a connecting entity before permitting a modification, whether it restricts access to the tracing-log data, and whether an unauthorised write

attempt is visibly rejected. OWASP ZAP is an active web-application scanner that probes an HTTP interface for common vulnerabilities and misconfigurations, including access-control and privilege-related issues [70]. Hydra is a network-authentication brute-force tool that attempts credential-based logon against an exposed authentication endpoint [71]. OWASP ZAP (CM-004) actively scans the IoT-Core web interface. Hydra (CM-003) is not applicable because the device exposes no authentication endpoint (Table 4.4). Under NV1, a write probe posts to three IoT-Core nodes. The unauthenticated configuration-protocol reset and the physical-barrier scenarios were designed but not executed. The activity reported below confirms a single anonymous read-only role and shows that three specific write requests were rejected, but the rejection does not distinguish input validation from an authorisation decision.

Access Control and Configuration Interfaces This activity examined the access-control model of the non-safety interface. It relates to CM-003 and CM-004, to requirements RQ-011, RQ-014, and RQ-009, and to assets B and G. It targets the IoT-Core interface and the configuration protocol and was run under NV1. The commands are listed in Appendix D.8.

The IoT-Core interface provided anonymous read access without a login. CM-003 was not applicable because no authentication endpoint was exposed. No Hydra execution is claimed. An active web scan was run with OWASP ZAP through a local proxy, and the report was saved. A comparison of multiple user roles was not applicable because the interface exposes a single anonymous read-only role.

A write probe was run against three IoT-Core nodes. A POST request to the safety-address node, to a digital-output node, and to the application-tag node was rejected in each case, and the responses are stored as evidence. The device answered with a bad-request response rather than an explicit authorisation denial. A bad-request response shows that the write was not accepted, but it does not by itself prove an access-control decision because it can also result from the request format. For the application-tag node, which the device documentation lists as writable, an explicit authorisation denial would have been the expected response.

The anonymous read-only access is an observation about the IoT-Core interface. It does not, on its own, decide RQ-011 or RQ-014. The three submitted write requests did not result in an accepted modification, but the returned bad-request responses do not distinguish input validation from an authorisation decision. The activity therefore does not demonstrate an access-control mechanism. It does not cover the full write surface,

the configuration-protocol behaviour, or the factory-reset path, which were not exercised. The tracing-log access control of RQ-014 is not evidenced beyond the single anonymous read-only role. The requirements therefore remain **Not Assessed** under the conservative aggregation rule.

4.3.7. Firmware and Configuration Integrity

Section 1.1.9 requires that hardware components transmitting signal or data relevant to connection or access to safety-critical software be protected against corruption (the parameter-integrity aspect of RQ-002) and that identified safety-critical software and data be protected against corruption (RQ-005). This category therefore examines whether firmware and configuration data are protected against corruption or forgery, whether an unsigned or modified firmware package can be installed, and whether the physical address and mode interface resists tampering. None of the ten company tests targets this category directly (Table 4.5). The abstract catalogue’s parameter-integrity rejection and forgery scenarios, the unsigned and modified firmware installation scenario, the update-package static-analysis scenario, the analytical safety-self-test scenario, and the physical tamper-resistance scenario each require the engineering tool, a spare unit for a risk-bearing firmware test, the NV3 in-path position, or physical access to the sealed interface. None of these scenarios was executed in the practical phase, so RQ-002 and RQ-005 have no stored device evidence. Both are reported as Not Assessed in Table 4.7, and the parameter-integrity, firmware-authenticity, and physical-tamper gaps identified during the theoretical validation remain open items for a future test campaign. CM-002, CM-006, and CM-008 are not applicable or blocked, as recorded in Table 4.4, and consequently do not contribute evidence to this or any other category.

4.4. Requirement Assessment Results

The purpose of this section is to show that the methodology can assign a traceable verdict to each derived component-level requirement from the collected evidence. The per-requirement results are the output of applying the method to one device under the NV1 position. They are not a conformance audit of the product, and no legal conformity is claimed. A Fail verdict denotes an evidenced absence of component capability against a derived requirement, not a declaration of legal non-conformity of the product. Under the allocation defined in Section 3.2.2, the corresponding machine-level obligation may instead

be satisfied by a compensating countermeasure at system or zone level, which the machine manufacturer or integrator must document. Section 4.4.1 relates the executed activities of Section 4.3 to the fourteen derived requirements and assigns a verdict according to the scale of Section 3.7.1. Section 4.4.2 summarises the resulting coverage of the practical phase as direct counts.

4.4.1. Verification Status per Requirement

All executed activities reported here used the network variant NV1. Every verdict is assessed against the SL-C 1 baseline established in Section 3.3.1 as the pragmatic minimum for an isolated component evaluation. A Fail verdict therefore indicates that the component does not meet this foundational capability level, not a failure against a higher target level. A verdict is assigned only where a stored artefact supports it. The execution status of a test and its verdict are distinct. An executed test can still leave the requirement Not Assessed when the complete acceptance condition is not covered. A definitive Fail can result from an evidenced device deviation even though the test itself succeeded. Where the evidence documents such a deviation, the requirement is assessed as Fail because the component capability is not demonstrated against the derived requirement. Table 4.7 summarises the current status per requirement. No requirement is verified as fully met. No legal conformity is claimed. Incomplete or ambiguous evidence is reported as Not Assessed under the conservative aggregation rule of Section 3.7.2.

Table 4.7.: Current requirement verification status under NV1

Req.	Supporting activity	Assessment	Limitation
RQ-001	attack-surface enumeration (Section 4.3.1)	Not Assessed	non-safety path only; no safety-channel or complete domain-separation evidence
RQ-002	none executed	Not Assessed	requires the engineering tool or the relevant NV2/NV3 scenario
RQ-003	intervention evidence and time attribution (Section 4.3.3)	Fail	no dedicated tracing log; no absolute timestamp or actor identity; compensating control at system level required

Continued on next page

Table 4.7.: Current requirement verification status under NV1 (continued)

Req.	Supporting activity	Assessment	Limitation
RQ-004	identification inventory (Section 4.3.1)	Not Assessed	product-level identification was observed, but complete software and data coverage was not established; cleartext is non-scoring evidence
RQ-005	none executed	Not Assessed	firmware and integrity tests not executed
RQ-006	installed-software inventory (Section 4.3.1)	Not Assessed	top-level firmware and bootloader versions are identified, but third-party components necessary for safe operation, such as the communication stack and the real-time operating system, remain unidentified
RQ-007	identification availability; connection-hold and load tests (Sections 4.3.1, 4.3.4)	Not Assessed	the identification read succeeded at every point tested, but availability was demonstrated only at one endpoint and outside the specified two-connection envelope, so availability at all times is not established across every permitted operating condition
RQ-008	software-event evidence baseline (Section 4.3.3)	Not Assessed	baseline evidence exists, but no software-installation event was induced
RQ-009	configuration evidence baseline (Section 4.3.3)	Not Assessed	baseline evidence exists, but no configuration change was induced
RQ-010	HTTP fuzzing; load test (Sections 4.3.5, 4.3.4)	Not Assessed	a diagnostic-interface DoS was demonstrated under the adversarial criterion, but the safety-channel effect requires NV2; the safety channel was not observed, the layer-two activity was not evidence-backed, and a fault-indication observation during fuzzing was not captured as a stored artefact

Continued on next page

Table 4.7.: Current requirement verification status under NV1 (continued)

Req.	Supporting activity	Assessment	Limitation
RQ-011	write probe on three nodes (Section 4.3.6)	Not Assessed	bad-request responses do not prove an access-control decision
RQ-012	cold-start deletion (Section 4.3.3)	Fail	substitute error log cleared by a cold restart; compensating control at system level required
RQ-013	none executed	Not Assessed	version history not exposed; current-version read does not evidence retention
RQ-014	access-control probe (Section 4.3.6); cleartext transmission observation (Section 4.3.2)	Not Assessed	single anonymous role; tracing-log access restriction and transport confidentiality were not fully assessed

The strongest evidence concerns the intervention log, whose detailed findings are reported in Section 4.3.3. The evidence supports the conclusion that the required tracing capability is not demonstrated and that the substitute artefact has the documented limitations behind RQ-003 and RQ-012. Both Fail verdicts are scoped to the interfaces examined under NV1: they record that the required tracing-log capability was not demonstrated through the IoT-Core diagnostic path, and they do not exclude that such a log could be exposed through another interface, such as the engineering tool, whose read operations were not part of the evidence-backed execution subset (Section 4.1.2). The identification tests demonstrate product-level information retrieval, but they do not establish complete software or data coverage. RQ-006 remains open for the same reason. Top-level firmware is identified, but third-party components necessary for safe operation, such as the communication stack and the real-time operating system, are not. RQ-007 is a distinct, availability-only question. Whether the identification information can be retrieved at all times was tested through the load and connection-hold activity, not through a version history. The absence of a version history instead limits RQ-013. Cleartext HTTP is recorded as a defence-in-depth observation and does not itself verify RQ-004 or RQ-014. RQ-013 is Not Assessed because a current-version read does not evidence retention of uploaded software versions. The robustness and load observations do not provide a complete RQ-010 assessment because the safety channel was not observed. The write probe shows only that three specific write

requests were not accepted because the device answered with a bad-request response, it does not establish an access-control decision. The remaining requirements are Not Assessed because their complete acceptance conditions were not covered under NV1.

4.4.2. Coverage of the Practical Phase

The assessment model of Section 3.7 is qualitative, so the results are summarised as direct counts rather than as ratios. Fourteen requirements were derived from the Regulation. Of the ten company tests, four were applicable and executed, two were applicable with adaptation, three were not applicable to the device or topology, and one was blocked in the laboratory. The two adapted procedures (CM-004 and CM-005) were executed against the related interface actually present and yielded a usable interface observation rather than a direct security verdict. An activity is counted once only when a stored artefact supports its interpretation. Unsupported observations and planned procedures are excluded. Two requirements, RQ-003 and RQ-012, are assessed as Fail on the basis of evidenced component-capability deviations. The remaining requirements are Not Assessed because their complete acceptance conditions were not covered. The evidence base includes raw Nmap and Nikto output, SSLscan output, inventory reads, error-log and system-tick reads, Siege summaries, connection-hold output, write-probe responses, boofuzz session data, and the OWASP ZAP report. These artefacts support reproducible observations, but not necessarily complete requirement verification. The limited number of definitive verdicts reflects the bounded scope of the practical phase. The assessment can be extended when the outstanding NV2, NV3, firmware, version-history, and engineering-tool activities are executed and recorded.

4.5. Discussion of Results

Section 4.4 assigned a verdict to each of the fourteen derived requirements. This section steps back from the individual verdicts and interprets the resulting pattern as a whole.

4.5.1. Synthesis of Findings Across Test Categories

Table 4.7 shows a pattern rather than an isolated result. Two requirements are assessed as Fail, and the remaining twelve are Not Assessed. Under the conservative aggregation rule

of Section 3.7.2, a Not Assessed verdict does not indicate that a security function is absent. It indicates that the executed evidence does not cover the complete acceptance condition of the requirement. The predominance of this verdict is therefore not primarily a finding about the device, but a direct consequence of the scope decisions documented in Section 4.1.3. The practical phase realised only the direct-access variant NV1, while a majority of the abstract test scenarios in Section 4.3 that could have produced a Pass or a further Fail depend on NV2, NV3, an induced software or configuration change, or the engineering tool. This is visible category by category. In identification and asset discovery (Section 4.3.1), top-level firmware was identified, but third-party components necessary for safe operation, such as the communication stack and the real-time operating system, remain unidentified, so complete software coverage was not established for RQ-004 and RQ-006. RQ-007's related question is distinct and availability-only. Whether the identification information can be retrieved at all times was tested through the load and connection-hold activity of Section 4.3.4, but only at one endpoint outside the specified operating envelope. In availability and resilience and in robustness (Sections 4.3.4, 4.3.5), the connection-hold procedure demonstrated a trivially reproducible denial of the diagnostic interface under the adversarial criterion, and the fuzzing activity produced further observations, but neither could be related to the cyclic safety channel, which only NV2 exposes, so RQ-010 remains Not Assessed at the requirement level. In authentication and access control (Section 4.3.6), the write probe demonstrated that three specific requests were rejected, but a bad-request response does not distinguish input validation from an authorisation decision, so the requirement remains open rather than confirmed.

The two Fail verdicts, RQ-003 and RQ-012, stand apart from this pattern because they were fully assessable within NV1 alone. Both concern the same substitute artefact, the IoT-Core error log, and both are demonstrated by the same evidence sequence in Table 4.6. The log carries only a relative system tick and no actor identity, and its content is deleted by a cold restart. This is the strongest finding of the practical phase, precisely because it did not require passive observation of the safety channel or an in-path position to demonstrate. It illustrates a general property of the methodology. A requirement can be conclusively assessed under a restricted network position when the demonstrating property lies entirely on the non-safety, IP-reachable path. Conversely, requirements whose acceptance condition touches the cyclic PROFINET/PROFIsafe communication, such as the domain-separation aspect of RQ-001 or the resilience aspect of RQ-010, cannot be conclusively assessed under NV1 regardless of how many NV1-compatible tests are executed against them because the observation point itself is insufficient for the claim.

The unrealised NV2 and NV3 variants therefore do not merely reduce the number of executed test cases. They remove the observation points required for entire classes of claims,

specifically those concerning the safety-relevant process data and in-path manipulation. This distinguishes two different reasons for a Not Assessed verdict that the aggregation rule of Section 3.7.2 treats alike but that carry different implications for future work. A requirement can be open because the induced event was never triggered under an otherwise sufficient access position (for example RQ-008 and RQ-009, where only baseline evidence was collected), or it can be open because the required access position itself was never established (for example the domain-separation aspect of RQ-001 and RQ-010). The first class can be closed by extending the test execution within the existing NV1 topology. The second class requires the topology itself to be extended, specifically a mirror or SPAN port for NV2 or an inline bridge for NV3, as noted in Section 4.1.3.

4.5.2. Answering the Research Questions

RQ1: How can the machine-level cybersecurity obligations of Regulation (EU) 2023/1230 be translated into verifiable, derived component-level security tests for a safety-related industrial gateway?

The instantiation in Section 4.2.1 demonstrates that the derivation chain defined in Section 3.3.1, from the Regulation through prEN 50742 to IEC 62443-4-2, and further through the asset and threat mapping of Section 3.3.2, produces requirements that are concrete enough to bind to a specific interface, asset, and network access variant on a real device (Table 4.3). The subsequent execution in Section 4.3 confirms that each requirement can be operationalised into at least one test activity, even where the activity ultimately yields a Not Assessed verdict rather than a Pass. The two Fail verdicts, RQ-003 and RQ-012, show that the chain is capable of producing a conclusive negative verdict when the acceptance condition lies within the realised access position. RQ1 is therefore answered affirmatively within the scope of NV1. The translation from a machine-level obligation to a verifiable, evidence-based component test is demonstrated end to end, though the demonstration is fully conclusive only for the subset of requirements whose acceptance condition does not depend on NV2 or NV3.

RQ2: Which tests of the existing company framework are applicable to an industrial gateway, and how can non-applicable tests be identified before execution?

Table 4.4 answers this question directly. Of the ten company tests, four were applicable and executed (CM-001, CM-007, CM-009, CM-010), two were applicable with adaptation

(CM-004, CM-005), three were not applicable to the device or the topology (CM-002, CM-003, CM-006), and one was blocked by the laboratory environment (CM-008). The applicability assessment of Section 4.2.2 identified all three not-applicable cases before execution, on the basis of an explicit tool-to-target check. CM-002 assumes a second network zone that the flat topology does not provide, CM-003 assumes an authentication endpoint that the SUT does not expose, and CM-006 assumes an operating-system shell that the embedded device does not expose. RQ2 is therefore answered by the methodology itself. A systematic applicability assessment, performed prior to execution and grounded in the assumed interface of each tool rather than in the tool’s general purpose, correctly separates applicable tests from tests whose assumed target interface is known in advance to be absent. This distinction is what prevents a not-applicable outcome from being misread as a passed or failed security test.

RQ3: What can direct-access testing at the physical IT/OT interface verify about the effectiveness of the implemented security functions, and where are its boundaries?

The synthesis in Section 4.5.1 bounds this answer. Direct-access testing under NV1 can conclusively verify requirements whose acceptance condition is exercised entirely through the IP-reachable non-safety path, as demonstrated by the Fail verdicts for RQ-003 and RQ-012. It can also produce reproducible interface-level observations, such as the absence of a TLS endpoint (Section 4.3.2) or the absence of an authentication endpoint (Section 4.3.6), that constrain but do not by themselves decide a requirement. Its boundary is equally precise: NV1 cannot observe the cyclic PROFINET/PROFIsafe communication between controller and SUT, so it cannot support any claim about the integrity of safety-relevant process data in transit, about in-path manipulation, or about the effect of a network disturbance on the safety function itself. Every requirement in Table 4.7 whose limitation column names the safety channel, the NV2 position, or the NV3 position marks exactly this boundary. RQ3 is therefore answered as follows: direct-access testing is sufficient to verify security functions that are fully exposed on the IT-side interface, but it structurally cannot verify or refute claims about the OT-side safety communication, regardless of the number or the depth of the tests executed at the IT/OT interface.

5. Summary and Outlook

5.1. Summary

This thesis addresses the absence of a systematic approach for verifying the security functions of industrial gateways at the physical IT/OT interface. With Regulation (EU) 2023/1230 making cybersecurity a mandatory part of the conformity assessment of machinery, established functional and safety validation processes lacked a requirement-driven method for verifying security functions. To bridge this gap, a prototype methodology was developed using the Design Science Research Methodology (DSRM). The objective was not to certify a product, but to develop and validate a systematic methodology for verifying derived component capabilities. The methodology derives required tests along a traceable chain, mapping machine-level obligations through the draft standard prEN 50742 to the component requirements of IEC 62443-4-2. Each requirement is linked to the affected assets, a STRIDE threat, and the relevant security function, with test cases specified on both a product-neutral scenario level and a concrete execution level. The approach reuses the existing company test framework through a four-outcome tool-to-target applicability assessment and classifies each test using the verification taxonomy of IEC 62443-4-1 Practice SVV.

The central steps of the methodology were demonstrated on a Safe Remote I/O (SRIO) device in an isolated laboratory under the direct-access network position NV1. The derivation yielded fourteen derived security requirements and decomposed the device into nine security-relevant assets, establishing a traceable derivation chain that answers the first research question. Answering the second research question, the applicability assessment classified the ten company tests before execution: four were applicable and executed (Nmap, Nikto, Siege, boofuzz), two were applicable with adaptation and executed against the interface actually present (OWASP ZAP, SSLscan), three were not applicable to the device or its flat topology (Nmap's zone-based procedure, Hydra, Lynis), and one was blocked in the laboratory (OpenVAS/GVM). The assessment therefore distinguished interfaces known to be absent in advance from interfaces whose absence was demonstrated by tool output.

During execution, a set of evidence-backed test activities characterised the reachable non-safety attack surface, the identification interface, the intervention log, and application-layer availability and robustness. Two requirements, RQ-003 and RQ-012, were assessed as Fail because no dedicated tracing log was exposed and the substitute artefact was neither attributable nor persistent across a power cycle. The remaining twelve requirements were assessed as Not Assessed, primarily because their complete acceptance condition depends on the passive-observation variant NV2, the in-path variant NV3, an induced software or configuration change, or the engineering tool, none of which were realised in this campaign. Additionally, a structured fuzzing run revealed a temporary, self-recovering disruption of the HTTP service, together with an unstored device fault indication, although its causation was not established. While the NV1 setup provided substantial evidence and answered the third research question in part, properties depending on cyclic safety communication require passive-observation (NV2) or in-path (NV3) variants.

Ultimately, the thesis contributes a requirement-derivation model translating machine-level obligations into derived component-level specifications, a traceability chain linking requirements to concrete evidence, a four-outcome tool-to-target applicability assessment, and a network-access-variant model clarifying test feasibility. A central methodological insight of this work is the necessary distinction between functional safety reactions and cybersecurity compliance. Successfully transitioning to a safe state following a fault does not inherently validate security-specific attributes such as accountability, integrity, or traceability, thereby necessitating independent verification. This distinction avoids conflating functional safety with cybersecurity conformity. Measured against the objective set out in the introduction, the results contribute to closing the identified gap. The methodology provides a traceable, evidence-based path from machine-level obligations to component-level verdicts, and the three research questions are answered within the bounded scope of the NV1 demonstration, even where individual verdicts remain Not Assessed rather than Pass or Fail. The predominance of Not Assessed verdicts is therefore a consequence of the deliberately bounded practical scope, not a failure of the methodology itself. For the industrial partner, the methodology is intended to support the process between regulatory expectations and functional testing by providing a reusable schema. Economically, the systematic verification may reduce additional tooling effort and is intended to support earlier identification of weaknesses, which is typically less costly to remediate before a product reaches the field than after a security-related recall or an incident response [72] [46, p. 67]. Ecologically, a reusable, requirement-driven test schema could reduce the need for repeated test setups and bespoke tooling across product variants, and earlier detection of security weaknesses may avoid unnecessary field replacements or hardware recalls, which together could lower the material and energy footprint of the verification process [73]. Socially, the methodology is intended to support the protection of

human operators because the Machinery Regulation treats cybersecurity as a prerequisite for functional safety, and an undetected security gap in a safety-related component can translate directly into a hazard for the people working with the machine [1, p. 4].

5.2. Outlook

To advance the current findings, the most immediate step is to realise the two remaining network access variants. Implementing the passive-observation variant NV2 using a mirroring switch or a network tap, and the in-path variant NV3 using a transparent bridge, would allow the execution of scenarios involving safety-telegram integrity, in-path manipulation, watchdog-passivation, and domain-separation. This is crucial for assessing requirements that depend on cyclic safety communication, in particular RQ-002 and RQ-005. In addition, outstanding activities requiring no additional variant should be completed within the same campaign, including the induced software-installation event for RQ-008, the configuration change for RQ-009, the TIA Portal-based identification and maintenance read operations for RQ-004, RQ-006, RQ-007, and RQ-013, the connection-hold procedure against the remaining identification endpoints, direct confirmation of the web-scanner Actuator finding, examination of the SNMP service, and reproduction of the fuzzing-induced HTTP disruption to establish causation. Together, these steps would extend the current partial coverage. Furthermore, applying the methodology to a device that exposes an authenticated and encrypted interface, such as over HTTPS, would additionally enable credential, transport-security, and access-control tests that were not applicable to the current device.

Beyond these immediate testing goals, the methodology itself can be extended. Because prEN 50742 grades the required test depth through the Safety Related Security Level (SRS�), the test catalogue can be expanded so that test strictness scales with the level of the affected asset and requirement-ranging, for example, from a simple checksum check at a low level to cryptographic verification under active manipulation at a high level [7, p. 21]. Another direction is the integration of machine learning. A profiling module that learns the normal network behaviour of a device in the testbed could significantly support the detection of anomalous behaviour during a test [74, pp. 12].

Finally, the methodology should be embedded in the development and quality-management processes. This includes assigning the responsibilities of the governance model described in Appendix C, fixing the point in the lifecycle at which tests are generated and evaluated, and integrating execution into a continuous integration and delivery pipeline for automated

security regression testing. It also requires maintaining the traceability matrix when the normative basis changes, such as when prEN 50742 is published in its final form. The allocation of evidentiary duties with the machine manufacturer should be confirmed when the standard reaches its final form.

Bibliography

Printed Sources

- [1] Europäisches Parlament and Rat der Europäischen Union, *Verordnung (eu) 2023/1230 des europäischen parlaments und des rates vom 14. juni 2023 über maschinen und zur aufhebung der richtlinie 2006/42/eg des europäischen parlaments und des rates sowie der richtlinie 73/361/ewg des rates*, 14/06/2023. Accessed: 8th Jul. 2026. [Online]. Available: <https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX:32023R1230> (cit. on pp. iii sq., 1, 14 sq., 21 sq., 71).
- [2] I. C. Ehie and M. A. Chilton, 'Understanding the influence of it/ot convergence on the adoption of internet of things (iot) in manufacturing organizations: An empirical investigation,' *Computers in Industry*, vol. 115, p. 103 166, 2020, ISSN: 0166-3615. DOI: <https://doi.org/10.1016/j.compind.2019.103166> [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0166361519307663> (cit. on pp. 1, 3 sq.).
- [3] P. K. Garimella, 'It-ot integration challenges in utilities,' in *2018 IEEE 3rd International Conference on Computing, Communication and Security (ICCCS)*, 2018, pp. 199–204. DOI: 10.1109/CCCS.2018.8586807 (cit. on pp. 1, 3, 5).
- [4] European Union Agency for Cybersecurity (ENISA), 'Good practices for security of internet of things in the context of smart manufacturing,' ENISA, Report, 11/2018. [Online]. Available: <https://www.enisa.europa.eu/publications/good-practices-for-security-of-iot> (cit. on p. 1).
- [6] TÜV Rheinland, 'Functional safety and cybersecurity - faq,' TÜV Rheinland, FAQ Document, 2020. [Online]. Available: https://www.tuv.com/content-media-files/master-content/global-landingpages/images/functional-safety-meets-cybersecurity/tuv_rheinland_de20_i07_2000498_en_faqs-fu%CC%88r-fscs_a4_web_final.pdf (cit. on p. 1).

- [7] *Sicherheit von maschinen - schutz vor korruption*, Deutsche und Englische Fassung prEN 50742:2025, Norm-Entwurf, CENELEC, 2025 (cit. on pp. 1, 9, 15 sq., 22 sq., 25, 71).
- [8] *IEC 62443-4-2:2019, IT-Sicherheit für industrielle Automatisierungssysteme - Teil 4-2: Technische Sicherheitsanforderungen an Komponenten industrieller Automatisierungssysteme (IACS)*, International Electrotechnical Commission, 2019 (cit. on pp. 1, 17 sq., 22, 25, 50, 54).
- [9] K. Peffers, T. Tuunanen, M. Rothenberger and S. Chatterjee, 'A design science research methodology for information systems research,' *Journal of Management Information Systems*, vol. 24, pp. 45–77, 01/2007. [Online]. Available: https://www.researchgate.net/publication/284503626_A_design_science_research_methodology_for_information_systems_research (cit. on pp. 2, 19).
- [10] W. Babel, in *Systemintegration in Industrie 4.0 und IoT: Vom Ethernet bis hin zum Internet und OPC UA*. Wiesbaden: Springer Fachmedien Wiesbaden, 2024, ISBN: 978-3-658-42987-4. DOI: 10.1007/978-3-658-42987-4_7 [Online]. Available: https://doi.org/10.1007/978-3-658-42987-4_7 (cit. on pp. 3, 6).
- [12] J. Hicking, M.-F. Stroh and S. Kremer, in *Smart and Sustainable Collaborative Networks 4.0*, L. M. Camarinha-Matos, X. Boucher and H. Afsarmanesh, Eds., Cham: Springer International Publishing, 2021 (cit. on p. 4).
- [15] H. Gsell and A. Rajchowski, in *Software-Technologien und IT-Systeme für die Wohnungs- und Immobilienwirtschaft: Künstliche Intelligenz, Digitale Plattformen, Property Technologies, IT-Ökosysteme*. Wiesbaden: Springer Fachmedien Wiesbaden, 2026, ISBN: 978-3-658-51129-6. [Online]. Available: https://doi.org/10.1007/978-3-658-51129-6_4 (cit. on p. 6).
- [18] W. Babel, 'Ethernetbasierte feldbusse und bussysteme,' in *Systemintegration in Industrie 4.0 und IoT: Vom Ethernet bis hin zum Internet und OPC UA*. Wiesbaden: Springer Fachmedien Wiesbaden, 2024, pp. 435–505. [Online]. Available: https://doi.org/10.1007/978-3-658-42987-4_10 (cit. on p. 6).
- [23] D. Craigen, N. Diakun-Thibault and R. Purse, 'Defining cybersecurity,' *Technology Innovation Management Review*, vol. 4, pp. 13–21, 10/2014, ISSN: 1927-0321. DOI: <http://doi.org/10.22215/timreview/835> [Online]. Available: <http://timreview.ca/article/835> (cit. on p. 7).
- [25] D. Vasović, G. Janačković, Ž. Vranjanac, S. Stamenković and B. Vasović, 'Enhancing CIA triad—confidentiality, integrity and availability in educational information systems through next-generation ISO/IEC 27001:2022-aligned security model,' *Applied Sciences*, vol. 16, no. 12, 2026. [Online]. Available: <https://www.mdpi.com/2076-3417/16/12/6260> (cit. on p. 7).

- [26] A. Khanna, 'Introduction to cybersecurity,' in *Securing an Enterprise : Maximizing Digital Experiences through Enhanced Security Measures*. Berkeley, CA: Apress, 2024, pp. 3–18. [Online]. Available: https://doi.org/10.1007/979-8-8688-1029-9_1 (cit. on p. 7).
- [27] *IEC 62443-3-3:2013, Industrielle Kommunikationsnetze - IT-Sicherheit für Netze und Systeme - Teil 3-3: Systemanforderungen zur IT-Sicherheit und Security-Level*, International Electrotechnical Commission, 2019 (cit. on pp. 7, 9, 17, 22, 25).
- [28] R. do Carmo and A. Schlensog, in *Automotive Threat Analysis and Risk Assessment in Practice: A practical guide to TARA following the ISO/SAE 21434 standard for automotive embedded and IT/OT systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2025, pp. 37–51. [Online]. Available: https://doi.org/10.1007/978-3-662-69614-9_4 (cit. on pp. 8 sqq.).
- [29] M. Wurm, in *Automotive Cybersecurity: Security-Bausteine für Automotive Embedded Systeme*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2022, ISBN: 978-3-662-64228-3. DOI: 10.1007/978-3-662-64228-3_4 [Online]. Available: https://doi.org/10.1007/978-3-662-64228-3_4 (cit. on pp. 8 sqq., 100).
- [30] S. Liebl, L. Lathrop, U. Raithel, A. Aßmuth, I. Ferguson and M. Söllner, 'Analyzing the attack surface and threats of industrial internet of things devices,' 2024. arXiv: 2405.16318 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2405.16318> (cit. on pp. 8 sq.).
- [31] C. Theisen, N. Munaiah, M. Al-Zyoud, J. C. Carver, A. Meneely and L. Williams, 'Attack surface definitions: A systematic literature review,' *Information and Software Technology*, vol. 104, pp. 94–103, 2018, ISSN: 0950-5849. DOI: <https://doi.org/10.1016/j.infsof.2018.07.008> [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584918301514> (cit. on p. 9).
- [32] J.-L. Richier, 'Security testing techniques,' Technical report, DIAMONDS Consortium, 2011. Deliverable ID, Tech. Rep., 2011 (cit. on pp. 9, 11, 13, 29).
- [36] O. B. Tauqeer, S. Jan, A. O. Khadidos, A. O. Khadidos, F. Q. Khan and S. Khattak, 'Analysis of security testing techniques,' *Intelligent Automation & Soft Computing*, vol. 29, no. 1, pp. 291–306, 2021. [Online]. Available: <https://pdfs.semanticscholar.org/2aa1/9e333637ff153803c4fbdf42fdad937e895e.pdf> (cit. on pp. 10 sq., 29).
- [37] B. Barz, in *Security-System Engineering: Ein Ansatz zu den Grundlagen der Informationssicherheit*. Wiesbaden: Springer Fachmedien Wiesbaden, 2025. DOI: 10.1007/978-3-658-48459-0_3 [Online]. Available: https://doi.org/10.1007/978-3-658-48459-0_3 (cit. on p. 10).

- [38] B. Potter and G. McGraw, 'Software security testing,' *IEEE Security & Privacy*, vol. 2, no. 5, pp. 81–85, 2004. DOI: 10.1109/MSP.2004.84 (cit. on pp. 11, 23, 26 sq., 29).
- [39] <. href="/contents.html?query=Firesmith">Donald Firesmith, 'Engineering security requirements,' *Journal of Object Technology*, vol. 2, no. 1, pp. 53–68, 01/2003, (column), ISSN: 1660-1769. DOI: 10.5381/jot.2003.2.1.c6 [Online]. Available: http://www.jot.fm/contents/issue_2003_01/column6.html (cit. on p. 11).
- [40] K. Scarfone, M. Souppaya, A. Cody and A. Orebaugh, 'Technical guide to information security testing and assessment,' National Institute of Standards and Technology, NIST Special Publication 800-115, 09/2008. Accessed: 21st Jul. 2026. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/115/final> (cit. on pp. 11, 28, 31, 33, 35 sq.).
- [41] N. T. Cong, L. Toan and T. Minh, 'An overview of static and dynamic analysis in application security testing,' *Journal of Military Science and Technology*, vol. 99, no. 99, pp. 1–11, 2024 (cit. on pp. 11 sq.).
- [43] *DIN EN IEC 62443-4-1 (VDE 0802-4-1):2018-10: IT-Sicherheit für industrielle Automatisierungssysteme - Teil 4-1: Anforderungen an den Lebenszyklus für eine sichere Produktentwicklung*, Deutsches Institut für Normung e. V. / VDE, 10/2018 (cit. on pp. 13, 17, 26, 28 sq., 31, 100, 102).
- [44] C. Poppe, 'Neue eu-maschinenverordnung: Von der richtlinie zur verordnung,' *CITplus*, vol. 28, no. 10, pp. 32–35, 2025. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/citp.202501012> (cit. on p. 14).
- [45] 'Ics security compendium,' Bundesamt für Sicherheit in der Informationstechnik (BSI), Bonn, Germany, Tech. Rep., 23/04/2024. Accessed: 20th Aug. 2026. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/ICS/ICS-Security_kompendium_pdf.html (cit. on pp. 22, 35 sq.).
- [46] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule and M. Thompson, 'Guide to operational technology (OT) security,' National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication (SP) 800-82 Rev. 3, 09/2023. DOI: 10.6028/NIST.SP.800-82r3 [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/82/r3/final> (cit. on pp. 22, 35, 70).
- [48] DIN Deutsches Institut für Normung e. V., *Informationssicherheit, cybersicherheit und schutz der privatsphäre – evaluationskriterien für it-sicherheit – teil 1: Einführung und allgemeines modell (iso/iec 15408-1:2022)*, 01/2024. Accessed: 15th Aug. 2026. [Online]. Available: <https://www-nautos-de.ezproxy-dhrv-1.redi-bw.de/MM3/search/item-detail/DE30100313> (cit. on p. 26).

- [53] 'Security testing framework: A kali linux toolset for verifying countermeasures cm-001 to cm-010,' English, ifm, Technical Report, internal (cit. on p. 30).
- [54] L. Pilorget, in *Testen von Informationssystemen: Integriertes und prozessorientiertes Testen*. Wiesbaden: Vieweg+Teubner Verlag, 2012. DOI: 10.1007/978-3-8348-8677-4_3 [Online]. Available: https://doi.org/10.1007/978-3-8348-8677-4_3 (cit. on p. 32).
- [55] J. Feld, 'Profinet - scalable factory communication for all applications,' in *IEEE International Workshop on Factory Communication Systems, 2004. Proceedings.*, 2004, pp. 33–38. DOI: 10.1109/WFCS.2004.1377673 (cit. on p. 36).
- [58] F. Witte, in *Testmanagement und Softwaretest: Theoretische Grundlagen und praktische Umsetzung*. Wiesbaden: Springer Fachmedien Wiesbaden, 2019, ISBN: 978-3-658-25087-4. DOI: 10.1007/978-3-658-25087-4_15 [Online]. Available: https://doi.org/10.1007/978-3-658-25087-4_15 (cit. on p. 37).
- [59] A. Glossar, 'Istqb®/gtb standardglossar der testbegriffe,' *German Testing Board eV*, 2013. [Online]. Available: https://www.swisstestingboard.org/wp-content/uploads/2018/Documents/Glossar/Glossar%20Deutsch-Englisch%203_11.pdf (cit. on p. 38).
- [61] 'The digital platform for process automation,' PROFIBUS & PROFINET International (PI), White Paper, 2024. [Online]. Available: https://www.profibus.com/fileadmin/media/downloadsection/brochures/White_Paper_PN_in_PA_ENG_2024.pdf (cit. on p. 51).
- [63] E. Rescorla, 'The transport layer security (tls) protocol version 1.3,' Internet Engineering Task Force (IETF), Internet-Draft, 04/03/2024. [Online]. Available: <https://www.ietf.org/archive/id/draft-ietf-tls-rfc8446bis-10.html> (cit. on p. 53).
- [72] 'The economic impacts of inadequate infrastructure for software testing,' National Institute of Standards and Technology (NIST), Technical Report NIST Planning Report 02-3, 05/2002. [Online]. Available: <https://www.nist.gov/system/files/documents/2021/03/24/econImpactSumm.v23.pdf> (cit. on p. 70).
- [74] S. Siboni, V. Sachidananda, A. Shabtai and Y. Elovici, *Security testbed for the internet of things*, 2016. arXiv: 1610.05971 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/1610.05971> (cit. on p. 71).
- [75] K.-H. Niemann and J.-N. Puls, 'Musterprozess für einen sicheren produkt- entwicklungslebenszyklus nach iec 62443-4-1,' Fakultät I - Elektro- und Informationstechnik, Tech. Rep., 2023. DOI: 10.25968/opus-2935 (cit. on p. 100).

- [76] K.-R. Müller, 'Lebenszyklus - mit integrierter sicherheit,' in *IT-Sicherheit mit System: Integratives IT-Sicherheits-, Kontinuitäts- und Risikomanagement - Sichere Anwendungen - Standards und Practices*. Wiesbaden: Springer Fachmedien Wiesbaden, 2026, ISBN: 978-3-658-52418-0. DOI: 10.1007/978-3-658-52418-0_15 [Online]. Available: https://doi.org/10.1007/978-3-658-52418-0_15 (cit. on p. 100).
- [78] W.-y. Lee, S.-h. Lee, C. Jin and C.-t. Hyun, 'Development of the raci model for processes of the closure phase in construction programs,' *Sustainability*, vol. 13, no. 4, 2021, ISSN: 2071-1050. DOI: 10.3390/su13041806 [Online]. Available: <https://www.mdpi.com/2071-1050/13/4/1806> (cit. on p. 101).
- [79] M. Schmidt, *Ifm cybersecurity test process: Description of the ifm cybersecurity test process as part of the secure development lifecycle*, version 0.1, Internal document, not publicly available, ifm electronic, 06/2026 (cit. on p. 101).
- [80] H. Assal and S. Chiasson, 'Security in the software development lifecycle,' in *Fourteenth Symposium on Usable Privacy and Security (SOUPS 2018)*, Baltimore, MD: USENIX Association, 08/2018, pp. 281–296, ISBN: 978-1-939133-10-6. [Online]. Available: <https://www.usenix.org/conference/soups2018/presentation/assal> (cit. on p. 102).

Online Sources

- [5] M. Carl. 'Neue eu-maschinenverordnung gilt ab 2027: Übergang langfristig vorbereiten,' IHK Lippe zu Detmold, Accessed: 14th Jul. 2026. [Online]. Available: <https://www.ihk.de/lippe-detmold/hauptnavigation/beraten-und-informieren/innovation-und-digitalisierung/aktuelles2/neue-eu-maschinenverordnung-5683128> (cit. on pp. 1, 14).
- [11] 'Operational technology (ot): Definition, architektur & praxis,' HWI IT, Accessed: 16th Jul. 2026. [Online]. Available: <https://hwi-it.de/blog/operational-technology-ot-grundlagen/> (cit. on pp. 4 sq.).
- [13] 'It & ot convergence: What it is & why it matters,' Orise, Accessed: 14th Aug. 2026. [Online]. Available: <https://orise.com/it-ot-convergence-explained/> (cit. on pp. 4 sq.).
- [14] 'Gateway,' Elektronik-Kompendium, Accessed: 16th Jul. 2026. [Online]. Available: <https://www.elektronik-kompendium.de/sites/net/0901111.htm> (cit. on pp. 5 sq.).

- [16] T. Kleinbongartz. 'Übersetzer zwischen den welten: Transparenz vom kontrollzentrum bis zum feldgerät,' Accessed: 14th Aug. 2026. [Online]. Available: <https://prozess-technik.industrie.de/chemie/anlagen-chemie/uebersetzer-zwischen-den-welten/> (cit. on p. 6).
- [17] J. Hohn. 'Gateway: Definition + alles wissenswerte für 2026,' Placetel, Accessed: 16th Jul. 2026. [Online]. Available: <https://www.placetel.de/ratgeber/gateway> (cit. on p. 6).
- [19] 'PROFIsafe - Safety mit PROFINET,' PROFIBUS Nutzerorganisation e.V., Accessed: 13th Sep. 2026. [Online]. Available: <https://www.profibus.de/profisafe> (cit. on p. 6).
- [20] Z. El Intissar. 'Profisafe protocol: How it works & sil ratings,' SCADA Protocols, Accessed: 13th Sep. 2026. [Online]. Available: <https://scadaprotocols.com/profisafe-complete-technical-guide/> (cit. on p. 6).
- [21] 'Was ist cybersicherheit?' Accessed: 14th Aug. 2026. [Online]. Available: <https://www.microsoft.com/de-de/security/business/security-101/what-is-cybersecurity> (cit. on p. 7).
- [22] G. Lindemulder, M. Kosinski and A. Jonker. 'What is cybersecurity?' IBM, Accessed: 14th Aug. 2026. [Online]. Available: <https://www.ibm.com/think/topics/cybersecurity> (cit. on p. 7).
- [24] EICTA Content Team. 'What are the three goals of cybersecurity? the cia triad explained (2026),' Accessed: 19th Jul. 2026. [Online]. Available: <https://www.eicta.iitk.ac.in/knowledge-hub/cyber-security/three-goals-of-cybersecurity-cia-triad> (cit. on pp. 7 sq.).
- [33] A. Gomstyn and A. Jonker. 'Was ist ein angriffsvektor?' IBM Think, Accessed: 19th Jul. 2026. [Online]. Available: <https://www.ibm.com/de-de/think/topics/attack-vector> (cit. on p. 9).
- [34] 'Microsoft threat modeling tool threats, Stride model,' Microsoft Learn, Accessed: 20th Jul. 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats> (cit. on p. 9).
- [35] L. Conklin. 'Threat modeling process (historical),' OWASP Foundation, Accessed: 20th Jul. 2026. [Online]. Available: https://owasp.org/www-community/Threat_Modeling_Process (cit. on pp. 9 sq., 26).
- [42] 'Security testing - software testing,' GeeksforGeeks, Accessed: 21st Jul. 2026. [Online]. Available: <https://www.geeksforgeeks.org/software-testing/security-testing/> (cit. on p. 12).

- [47] 'Security functions,' SoftGuide, Accessed: 14th Sep. 2026. [Online]. Available: <https://www.softguide.com/function/security-functions> (cit. on p. 26).
- [49] 'OWASP IoT Security Testing Guide (ISTG),' Accessed: 20th Aug. 2026. [Online]. Available: <https://github.com/OWASP/owasp-istg> (cit. on p. 28).
- [50] 'Ics techniques,' Accessed: 17th Aug. 2026. [Online]. Available: <https://attack.mitre.org/techniques/ics/> (cit. on pp. 28 sq., 31, 33).
- [51] 'Ics tactics,' Accessed: 17th Aug. 2026. [Online]. Available: <https://attack.mitre.org/tactics/ics/> (cit. on p. 29).
- [52] T. Manana. 'Penetration testing methodologies: Owasp, ptes, osstmm, nist and mitre att&ck explained,' Hard2bit, Accessed: 15th Aug. 2026. [Online]. Available: <https://hard2bit.com/en/blog/penetration-testing-methodologies-owasp-ptes-osstmm-nist-mitre/> (cit. on pp. 29, 31, 33).
- [56] GeeksforGeeks. 'Top 10 alterantives of kali linux,' GeeksforGeeks, Accessed: 14th Sep. 2026. [Online]. Available: <https://www.geeksforgeeks.org/linux-unix/top-10-alterantives-of-kali-linux/> (cit. on p. 37).
- [57] g0tmi1k. 'What is kali linux? | kali linux documentation,' OffSec Services Limited, Accessed: 16th Aug. 2026. [Online]. Available: <https://www.kali.org/docs/introduction/what-is-kali-linux/> (cit. on p. 37).
- [60] 'Nmap.usage.txt,' Accessed: 10th Sep. 2026. [Online]. Available: <https://svn.nmap.org/nmap/docs/nmap.usage.txt> (cit. on p. 51).
- [62] C. Sullo and D. Lodge. 'Nikto wiki: Overview & description,' Accessed: 18th Sep. 2026. [Online]. Available: <https://github.com/sullo/nikto/wiki/Overview-&-Description> (cit. on p. 51).
- [64] rbsec. 'Sslscan: Fast ssl scanner,' Accessed: 17th Sep. 2026. [Online]. Available: <https://github.com/rbsec/sslscan> (cit. on p. 53).
- [65] 'Sslscan | kali linux tools.' Accessed: 2026-09-17. [Online]. Available: <https://www.kali.org/tools/sslscan/> (cit. on p. 53).
- [66] D. Song. 'Macof(8) - linux man page,' Accessed: 18th Sep. 2026. [Online]. Available: <https://linux.die.net/man/8/macof> (cit. on p. 56).
- [67] 'Hping3 | kali linux tools,' Accessed: 18th Sep. 2026. [Online]. Available: <https://www.kali.org/tools/hping3/> (cit. on p. 56).
- [68] J. Fulmer. 'Siege,' Accessed: 18th Sep. 2026. [Online]. Available: <https://github.com/JoeDog/siege> (cit. on p. 56).
- [69] 'Boofuzz quickstart - boofuzz documentation,' Accessed: 18th Sep. 2026. [Online]. Available: <https://boofuzz.readthedocs.io/en/stable/user/quickstart.html> (cit. on p. 58).

- [70] 'Getting started with zap,' Accessed: 10th Sep. 2026. [Online]. Available: <https://www.zaproxy.org/getting-started/> (cit. on p. 60).
- [71] 'Hydra | kali linux tools,' Accessed: 10th Sep. 2026. [Online]. Available: <https://www.kali.org/tools/hydra/> (cit. on p. 60).
- [73] 'Hardware-effizienz | learn green software,' Green Software Foundation, Accessed: 18th Sep. 2026. [Online]. Available: <https://learn.greensoftware.foundation/de/hardware-efficiency/> (cit. on p. 70).
- [77] Cyber Sierra. 'Security raci matrix: Who really owns what in your organization?' Accessed: 12th Aug. 2026. [Online]. Available: <https://www.cybersierra.co/blog/security-raci-matrix-guide> (cit. on pp. 101 sq.).

List of Abbreviations

AI	Artificial Intelligence 88
AIC	Availability, Integrity and Confidentiality 5
API	Application Programming Interface 28, 37
ARP	Address Resolution Protocol 36
CIA	Confidentiality, Integrity and Availability 5, 7, 8
COM	Communication Module 42, 43, 52
CPU	Central Processing Unit 55
CR	Component Requirement iii, iv, 17, 18, 23–26, 31–33, 50, 54, 57, 89, 95–99
CRC	Cyclic Redundancy Check 6, 94, 97
CRM	Customer Relationship Management 4
DAST	Dynamic Application Security Testing 12, 37
DCP	Discovery and Configuration Protocol 36, 42
DoS	Denial of Service 9, 17, 25, 32, 33, 56, 57, 63
DSRM	Design Science Research Methodology iii, iv, 2, 19, 20, 69
EDR	Embedded Device Requirement 89, 96–99
EN	European Standard 15, 16
ERP	Enterprise Resource Planning 4
FR	Foundational Requirement 17, 18, 25, 33, 34, 96–99

HMI	Human Machine Interface 4
HTML	Hypertext Markup Language 50
HTTP	Hypertext Transfer Protocol 5, 21, 28, 29, 31, 33, 35, 37, 42, 44, 45, 48, 50–53, 55–60, 64, 70, 71
HTTPS	Hypertext Transfer Protocol Secure 48, 71
I²C	Inter Integrated Circuit 28
I/O	Input/Output 3
IACS	Industrial Automation and Control Systems 7, 17
IAST	Interactive Application Security Testing 12
ICS	Industrial Control System 1, 28, 29, 31, 33
IEC	International Electrotechnical Commission iii, iv, vii, 1–3, 8, 10, 13, 16, 17, 20, 23–34, 46, 48, 59, 67, 69, 87, 89, 95–100, 102
IoT	Internet of Things 28, 29, 35, 41, 42, 44, 45, 49–54, 56, 58, 60, 64, 66
IP	Internet Protocol 5, 35, 36, 44, 45, 48, 57, 66, 68, 103
IT	Information Technology 1–7, 20, 21, 23, 25, 27, 28, 30, 42, 68, 69, 87
JSON	JavaScript Object Notation 51
JTAG	Joint Test Action Group 42
MAC	Media Access Control 58
ML	Maturity Level 17
NIST	National Institute of Standards and Technology 11, 28, 29, 31, 33, 35
NV1	Network Variant 1 iii, iv, 35, 41, 42, 44, 45, 48–54, 56–58, 60–62, 64–70, 86, 103
NV2	Network Variant 2 35, 36, 41, 44, 45, 49, 52, 53, 56–58, 63, 65–68, 70, 71, 86
NV3	Network Variant 3 36, 41, 44, 45, 49, 52, 58, 61, 65–68, 70, 71, 86
OSI	Open Systems Interconnection 6

OT	Operational Technology 1–6, 20, 21, 23, 25, 27, 29, 35, 42, 68, 69, 87
OWASP	Open Worldwide Application Security Project 28, 29, 31, 33, 48, 49, 60, 65, 69
PCB	Printed Circuit Board 8
PLC	Programmable Logic Controller 4, 43
PROFINET	Process Field Network 5, 6, 21, 28, 35, 41–43, 45, 66, 68
RACI	Responsible, Accountable, Consulted, Informed 101
RE	Requirement Enhancement 18, 89, 95–99
SAST	Static Application Security Testing 11
SBOM	Software Bill of Materials 50, 52
SCA	Software Composition Analysis 12
SCADA	Supervisory Control and Data Acquisition 4
SCPU	Safety CPU 42, 52
SDLC	Secure Development Life Cycle 17
SL	Security Level 17
SL-C	Security Level Capability iii, iv, 16, 18, 24, 62, 95
SNMP	Simple Network Management Protocol 51, 71
SPAN	Switched Port Analyzer 45, 67
SRIO	Safe Remote I/O iii, iv, vi, 1, 2, 21, 41–50, 55, 56, 69, 87, 103
SRSL	Safety Related Security Level 16, 23, 71, 93–96, 98
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege iii, iv, 2, 9, 10, 24, 26, 32, 45, 46, 69, 87, 89
SUT	System Under Test 1, 20–24, 30, 35, 36, 40, 41, 43–45, 48, 51, 68, 87, 89
SVV	Security Verification and Validation Testing 13, 27, 28, 30, 31, 33, 69, 102
SysCom	System Communication 42

TARA	Threat Analysis and Risk Assessment 9
TCP	Transmission Control Protocol 5, 48, 50, 51
TIA	Totally Integrated Automation 37, 43, 44, 71
TLS	Transport Layer Security 30, 48, 52, 53, 68
UART	Universal Asynchronous Receiver Transmitter 28
UDP	User Datagram Protocol 50, 51
VLAN	Virtual Local Area Network 43

List of Figures

3.1. Network Variant 1, direct access	35
3.2. Network Variant 2, passive observation	36
3.3. Network Variant 3, active in-path manipulation	36
4.1. Logical topology and addressing of the implemented test environment . .	44
C.1. High-level overview of the Secure Development Lifecycle	101

List of Tables

2.1. Differences between IT and OT environments	5
2.2. Mapping of STRIDE threat categories to cybersecurity Properties	10
2.3. Comparison of static analysis detection approaches	12
3.1. Mapping of STRIDE threat categories to security functions	26
3.2. Comparison of established security testing approaches	29
3.3. Mapping of security categories to IEC 62443-3-3 requirements	34
3.4. Mapping of security categories to regulatory requirements	34
4.1. Overview of assets and protection objectives	42
4.2. Components of the implemented test environment	43
4.3. Instantiated requirement-to-asset mapping for the SRIO	46
4.4. Applicability of the company framework tests to the SUT	48
4.5. Test methods and regulation requirements per security category	49
4.6. Recorded system tick values and error-log entry counts	55
4.7. Current requirement verification status under NV1	62
A.1. List of artificial intelligence based tools used	88
B.1. Security requirements derived from Regulation (EU) 2023/1230	89
B.2. Mapping of Machinery Regulation requirements to prEN 50742	93
B.3. Mapping of derived requirements to IEC 62443-4-2	96
C.1. RACI matrix for the verification lifecycle	102

A. Note on the Use of Artificial Intelligence

Within the scope of this thesis, tools based on Artificial Intelligence (AI) were used. Table A.1 provides an overview of the tools used and their respective purpose of application.

Table A.1.: List of AI-based tools used

Tool	Description of Use
DeepL	• Translation and drafting assistance (German <--> English) • Translation Abstract chapter
Gemini	• Assistance with understanding fundamental concepts • Finding Literature Sources
Microsoft Copilot 365	• Chapter structuring • Drafting assistance • Grammar and spelling checks • Formatting (bullet points, tables, formulas) in LaTeX (overall) • Generation of LaTeX code for Figures 4.1, 3.1, 3.2, 3.3 based on text descriptions • Assistance with understanding fundamental concepts • Code ideas / error analysis for test cases
Github Copilot	• LaTeX code: Tables • Abbreviations • Formatting (bullet points, tables, formulas) in LaTeX (overall)

B. Detailed Requirement Derivation Matrices

This appendix provides the complete, requirement-by-requirement documentation underlying the derivation methodology described in Section 3.3.1. It contains 14 requirements derived from obligations addressed to machinery and related products under **Regulation (EU) 2023/1230**. They are expressed here as derived component-level requirements for the SUT for verification purposes. They do not state that the Regulation applies directly to the component. The requirements are mapped to **prEN 50742** and, subsequently, to **IEC 62443-4-2** CRs, REs, and component-type-specific instantiations such as Embedded Device Requirements (EDRs).

B.1. Security Requirements Derived from Regulation (EU) 2023/1230

Table B.1 lists the 14 derived component-level requirements originating from Annex III, Sections 1.1.9 and 1.2.1 of Regulation (EU) 2023/1230. Each entry records the originating source clause, the requirement statement, and the assigned Security Objective. The Security Objective column records the primary protection goal of each requirement only. A requirement may additionally engage further objectives through its associated threats, as reflected in the STRIDE mapping of Table 4.3.

Table B.1.: Security requirements derived from Regulation (EU) 2023/1230

Field	Description
Requirement RQ-001	
Source	Annex III, Section 1.1.9
Requirement	The system shall be designed so that the connection of another device, including any remote device that communicates with it, does not lead to a hazardous situation.

Table B.1 continued from previous page

Field	Description
Security Objective	Integrity
Requirement RQ-002	
Source	Annex III, Section 1.1.9
Requirement	The system shall protect hardware components transmitting signal or data relevant to connection or access to safety-critical software against corruption, whether accidental or intentional.
Security Objective	Integrity
Requirement RQ-003	
Source	Annex III, Section 1.1.9
Requirement	The system shall collect evidence of any legitimate or illegitimate intervention in hardware components relevant for connection or access to safety-critical software.
Security Objective	Accountability
Requirement RQ-004	
Source	Annex III, Section 1.1.9
Requirement	The system shall identify the software and data that are critical for compliance with the essential health and safety requirements.
Security Objective	Integrity
Requirement RQ-005	
Source	Annex III, Section 1.1.9
Requirement	The system shall protect identified safety-critical software and data against corruption, whether accidental or intentional.
Security Objective	Integrity
Requirement RQ-006	
Source	Annex III, Section 1.1.9
Requirement	The system shall identify the software installed on it that is necessary for safe operation.
Security Objective	Integrity

Table B.1 continued from previous page

Field	Description
Requirement RQ-007	
Source	Annex III, Section 1.1.9
Requirement	The system shall be able to provide the identification information of safety-relevant installed software at all times in an easily accessible form.
Security Objective	Availability
Requirement RQ-008	
Source	Annex III, Section 1.1.9
Requirement	The system shall collect evidence of any legitimate or illegitimate intervention in the software installed on it.
Security Objective	Accountability
Requirement RQ-009	
Source	Annex III, Section 1.1.9
Requirement	The system shall collect evidence of any modification of the installed software or its configuration.
Security Objective	Accountability
Requirement RQ-010	
Source	Annex III, Section 1.2.1
Requirement	The control system shall withstand reasonably foreseeable malicious attempts from third parties that could lead to a hazardous situation.
Security Objective	Availability
Requirement RQ-011	
Source	Annex III, Section 1.2.1
Requirement	The system shall prevent modifications to safety-relevant settings or rules, including those generated during a learning phase, where such modifications could lead to a hazardous situation.
Security Objective	Integrity

Table B.1 continued from previous page

Field	Description
Requirement RQ-012	
Source	Annex III, Section 1.2.1
Requirement	The system shall enable a tracing log of data generated in relation to an intervention for five years after the machinery or related product has been placed on the market or put into service.
Security Objective	Accountability
Requirement RQ-013	
Source	Annex III, Section 1.2.1
Requirement	The system shall enable a tracing log of the versions of safety software uploaded after the machinery or related product has been placed on the market or put into service, for five years after each upload.
Security Objective	Accountability
Requirement RQ-014	
Source	Annex III, Section 1.2.1
Requirement	The system shall restrict access to the tracing log data exclusively to demonstrating conformity further to a reasoned request from a competent national authority.
Security Objective	Confidentiality

B.2. Mapping to prEN 50742

Table B.2 maps the 14 requirements of Table B.1 onto the corresponding clauses of **prEN 50742** (Safety of machinery - Protection against corruption). The mapping types are used as follows: **Partial** indicates that the clause addresses the obligation but does not fully cover it; **Supporting** indicates that the clause contributes context or a prerequisite but is not the operative requirement; and **No mapping** indicates that the draft standard explicitly excludes or does not address the obligation.

Table B.2.: Mapping of Machinery Regulation requirements to prEN 50742

RQ ID	prEN 50742 clause	prEN 50742 requirement/topic	Mapping type
RQ-001	§4.2; §4.3	Design/construction of machinery shall not permit hazardous situations arising from connections (physical, logical, indirect).	§4.2: Supporting §4.3: Supporting
RQ-002	§1; §3.5; §7.1; §7.2.1	Scope explicitly includes hardware components/interfaces (incl. remote-device interfaces) that transmit signals/data affecting machine safety; §7.1 requires identification and protection of accessible interfaces by compensating/countermeasures.	§1, §3.5, §7.2.1: Supporting §7.1: Partial
RQ-003	§3.4; §7.3.3; §7.4.3.4.5	“Evidence” definition includes physical tamper evidence (broken seals, damaged interface covers, e.g. RJ45); §7.3.3 requires physical evidence where digital evidence collection is not reasonably practicable; §7.4.3.4.5 is the only operative requirement mandating detection of physical manipulation (SRSL0-3).	§3.4: Supporting §7.3.3, §7.4.3.4.5: Partial
RQ-004	§3.12; §7.5	“Critical data” defined as data whose manipulation can increase hazard; §7.5 requires identification of software versions/configuration data relevant for protection against falsification.	§3.12: Supporting §7.5: Partial
RQ-005	§3.5; §7.1; §7.4.3.4.1	SRSL-graded software/information integrity requirements, from checksum verification at start-up (SRSL1) to cryptographic hash/HMAC/CMAC verification during operation (SRSL3).	§3.5, §7.1: Supporting §7.4.3.4.1: Partial
RQ-006	§7.5; §9	Identification of software versions/configuration data “relevant for protection against falsification” must be available; §9 additionally requires that user information include the identification of software versions and configuration data relevant for safety functions.	§7.5: Partial §9: Supporting

Table B.2 continued from previous page

RQ ID	prEN 50742 clause	prEN 50742 requirement/topic	Mapping type
RQ-007	§7.5; §9	Identification data “must be available on request”, in readable form, via an integrated system or an external tool; §9 requires user information to include instructions for accessing this data and its modifications. Note: prEN 50742 requires availability “on request” rather than “at all times” as mandated by the regulation, so the mapping remains partial.	§7.5: Partial §9: Supporting
RQ-008	§3.2; §7.3.1; §7.3.2; §7.3.3; §7.3.4	Enumerated intervention types with minimum recorded data; §7.3.4 specifies the log must at least include the records of the last intervention of each type.	§3.2: Supporting §7.3.1, §7.3.2, §7.3.3, §7.3.4: Partial
RQ-009	§3.2; §3.3; §7.3.1; §7.3.2; §7.3.3; §7.3.4	“Modification” is defined as a change caused by an “intervention”, incl. software version, sensor calibration, configuration data); §7.3.1 b)/c) require identification means (version number, CRC) for SRESW/SRASW updates; §7.3.4 specifies the associated minimum retention of at least five years per intervention type.	§3.2, §3.3: Supporting §7.3.1, §7.3.2, §7.3.3, §7.3.4: Partial
RQ-010	§4.3; §7.1; §7.2.1; §7.4.2; §7.4.3	Process principles: eliminate vulnerabilities that could lead to hazardous situations; otherwise mitigate; otherwise inform the user via user information of compensating measures; §7.1 requires identification and protection of accessible interfaces, §7.2.1 requires risk-based selection of security measures, and §7.4.2/§7.4.3 require determination and maintenance of SRESL-graded security requirements, all contributing to withstanding malicious third-party attempts.	§4.3, §7.2.1, §7.4.2: Supporting §7.1, §7.4.3: Partial

Table B.2 continued from previous page

RQ ID	prEN 50742 clause	prEN 50742 requirement/topic	Mapping type
RQ-011	§7.4.3.3.1; §3.2; §3.3	SRSL-graded authorisation requirements for interventions (SRSL1/2: authorisation required; SRSL3: privileged/role-based authorisation); the “intervention” definition names safety routines, safety variables, and calibration data as examples. Note: prEN 50742 contains no provision addressing learning-phase or self-evolving machinery behaviour; Annex ZZ correlation table (Table ZZ.1) does not correlate 1.2.1, Abs. 2, Point d) with any clause	§7.4.3.3.1: Partial §3.2, §3.3: Supporting
RQ-012	§7.3.1; §7.3.3; §7.3.4	Log must contain at least the evidence of the last intervention of each type; records retained at least five years; log active at the latest from the time the machinery is put into service.	§7.3.1, §7.3.3, §7.3.4: Partial
RQ-013	§7.3.1	Explicit exclusion: versions of SRASW/SRESW binaries need not be stored in the tracing log.	No mapping
RQ-014	§7.3.5; §7.3.3; §7.4.3.3.1	Logs must be “properly secured” against tampering; deletion only via an authorised procedure; digital evidence must be readable and accessible.	§7.3.5, §7.3.3: Supporting §7.4.3.3.1: Partial

Where the draft standard explicitly excludes an obligation, as for RQ-013, the derivation proceeds directly from the Regulation; the absence of a standard clause does not remove the underlying obligation.

B.3. Mapping to IEC 62443-4-2

Table B.3 extends the derivation chain from **prEN 50742** to IEC 62443-4-2. Where no **prEN 50742** clause is available, the mapping proceeds directly from **Regulation (EU) 2023/1230**. The mapped CRs are to be satisfied at the target Component SL-C declared in Section 3.3.1; applicable REs are included accordingly. IEC 62443-4-2 organises its CRs

under the seven FRs introduced in Section 2.4.3.

Table B.3.: Mapping of derived requirements to IEC 62443-4-2

RQ ID	FR	CR/RE/EDR	IEC 62443-4-2 topic	Mapping rationale	Type
RQ-001	FR3	CR 3.1	Communication integrity	CR 3.1 requires the component to verify the integrity of information transmitted over a communication channel, the closest component-level control for connection-related corruption.	Partial
RQ-002	FR3	CR 3.1; EDR 3.11	Communication integrity; Physical tamper resistance and detection (embedded device)	CR 3.1 covers the integrity of the information transmitted over such an interface, while EDR 3.11 covers protection of the interface hardware itself against unauthorised physical access and corruption.	Partial
RQ-003	FR2; FR3	CR 2.8; CR 2.12; EDR 3.11	Auditable events; Non-repudiation; Physical tamper resistance and detection (embedded device)	prEN §7.3.3 makes physical evidence (EDR 3.11) the fallback used only where digital evidence collection (CR 2.8) is not practicable. CR 2.8 supplies the primary digital-evidence/event-generation mechanism; CR 2.12 supplies the attribution property implied by “legitimate or illegitimate” (the evidence must be undeniably traceable to the causing action)	Partial
RQ-004	FR7	CR 7.8	Control system component inventory	CR 7.8’s component-inventory capability is the closest CR-level asset-identification control. – > IEC 62443-4-1: SUM-1/SM-9/SM-10	Supporting
RQ-005	FR3	CR 3.4; EDR 3.14	Software and information integrity; Integrity of the boot process (embedded device)	prEN’s SRSL-graded integrity framework (§7.4.3.4) covers general software/information integrity (§7.4.3.4.1 → CR 3.4) and, as a distinct sub-case, boot-process integrity (§7.4.3.4.2).	Partial

B. Detailed Requirement Derivation Matrices

Table B.3 continued from previous page

RQ ID	FR	CR/RE/EDR	IEC 62443-4-2 topic	Mapping rationale	Type
RQ-006	FR7	CR 7.8	Control system component inventory	CR 7.8's inventory capability is the closest CR-level match for tracking installed-software identity. -> IEC 62443-4-1: SUM-1/SM-9/SM-1	Supporting
RQ-007	FR7	CR 7.1, CR 7.2, CR 7.8	Control system component inventory	CR 7.8 supports retrieval of inventory information but does not mandate continuous, unsolicited availability, CR7.1 (DoS protection), CR7.2 Resource Management -> IEC 62443-4-1: SUM-1/SM-9/SM-1	Supporting
RQ-008	FR2	CR 2.8; CR 2.12	Auditable events; Non-repudiation	prEN §7.3.1/§7.3.2 anchor the enumerated intervention types and minimum recorded data to CR 2.8 (event generation); the "legitimate or illegitimate" attribution qualifier matches CR 2.12 (non-repudiation).	CR 2.8: Partial CR 2.12: Supporting
RQ-009	FR2; FR3	CR 2.8; CR 2.12; CR 3.4	Auditable events; Non-repudiation; Software and information integrity	prEN's "modification" definition and the version-number/CRC identification means combine an audit obligation (CR 2.8) with an integrity-verification obligation (CR 3.4). Because a modification is defined as a change caused by an intervention (§3.2), the attribution qualifier extends to modification evidence, which CR 2.12 (non-repudiation) addresses.	CR 2.8, CR 3.4: Partial CR 2.12: Supporting
RQ-010	FR7; (FR1; FR2; FR3)	CR 7.1 (1)	Protection against denial-of-service events	The regulation's requirement that control systems withstand reasonably foreseeable malicious third-party attempts corresponds, at the component level, to CR 7.1 (protection against denial-of-service events) together with its RE(1) (mitigation of communication-load flooding).	Partial

B. Detailed Requirement Derivation Matrices

Table B.3 continued from previous page

RQ ID	FR	CR/RE/EDR	IEC 62443-4-2 topic	Mapping rationale	Type
RQ-011	FR1; FR2; FR3	CR 1.1; CR 1.2; CR 2.1 (2); CR 3.4	Human/device identification and authentication; Authorisation enforcement (role-based); Software and information integrity	prEN's SRS�-graded authorisation requirement for interventions corresponds to CR 2.1, with role/task-based authorisation at SRS�3 (§7.4.3.3.1) matching CR 2.1's RE(2) (mapping of permissions to roles). CR 1.1 addresses authentication of the human-operator path, CR 1.2 (identification and authentication of software processes and devices) additionally applies to that path. CR 3.4 is included because "preventing modifications to safety-relevant settings or rules" is fundamentally a configuration-integrity property. The learning-phase/self-evolving-behaviour aspect has no counterpart in IEC 62443-4-2.	CR 1.1, CR 1.2, CR 3.4: Supporting CR 2.1 (2): Partial
RQ-012	FR2	CR 2.8; CR 2.9	Auditable events; Audit storage capacity	prEN's requirement that the log retain at least the evidence of the last intervention of each type combines the event-generation obligation of CR 2.8 with the storage-capacity obligation of CR 2.9. IEC 62443-4-2 defines no Component Requirement mandating a specific retention duration.	CR 2.8, CR 2.9: Partial
RQ-013	FR2	CR 2.8; CR 2.9	Auditable events (configuration changes); Audit storage capacity	CR 2.8's auditable-event categories include configuration changes and automation-system events, which are broad enough to cover the upload of a new safety-software version as a recordable event, and CR 2.9 provides the associated storage-capacity capability (without mandating the specific five-year, per-upload retention period).	CR 2.8, CR 2.9: Partial

B. Detailed Requirement Derivation Matrices

Table B.3 continued from previous page

RQ ID	FR	CR/RE/EDR	IEC 62443-4-2 topic	Mapping rationale	Type
RQ-014	FR2; FR3; FR4; FR6	CR 2.1; CR 3.9; CR 4.1; CR 6.1 (supporting)	Authorisation enforcement; Protection of audit information; Data confidentiality; Audit log accessibility (supporting)	CR 2.1 (authorisation enforcement) is the primary control restricting who may access the log and for what purpose, and CR 4.1 (confidentiality of information at rest for which read authorisation is explicitly required). CR 3.9 additionally protects the log against tampering and unauthorised deletion. CR 6.1 (ability to access event logs) is included only as a supporting control, since it ensures that an already-authorised party can retrieve the log on request.	CR 2.1: Partial CR 3.9, CR 4.1, CR 6.1: Supporting

C. Secure Development Lifecycle

Figure C.1 summarises a high-level reference model in which the activities shown are decomposed into further sub-activities. The model is based on the process-oriented security practices of IEC 62443-4-1 [43] and on complementary lifecycle-oriented security approaches [75].

The lifecycle demonstrates that cybersecurity is not limited to a single verification activity prior to product release. Instead, security is addressed throughout the development and operational lifecycle [75, pp. 5] [29, pp. 68]. The process begins with the definition of the scope and security objectives. A **TARA**, is performed to identify and assess relevant cybersecurity risks and to determine the required security level. Based on the assessment results, detailed countermeasures are derived and incorporated into the security architecture. The resulting architectural constraints provide input to the development phase, during which the specified security measures are implemented [75, pp. 8].

The implemented security measures are evaluated by means of static and dynamic analysis. The verification phase further comprises the definition and preparation of the security test strategy, the development of test cases, and the execution of the corresponding security tests. Identified findings are returned to the development phase for remediation and subsequent re-evaluation. The verification results provide the evidence required to support the release decision [76, pp. 788] [75, pp. 6, 16]. Following release, continuous monitoring is applied to identify newly discovered vulnerabilities, emerging threats, and security-relevant incidents. The resulting observations may initiate a reassessment or an incident-response process [75, pp. 18].

The dashed connections shown in Figure C.1 represent feedback paths and emphasise that the lifecycle is iterative rather than strictly sequential. Findings identified during security testing are returned to secure implementation for remediation. Architectural constraints may require the assumptions or results of the risk assessment to be reconsidered. In addition, information obtained through continuous monitoring may trigger a reassessment or incident response, thereby re-entering the lifecycle at the risk-assessment stage. These feedback paths support the continuous refinement of the risk assessment, security architecture, implementation, and verification activities

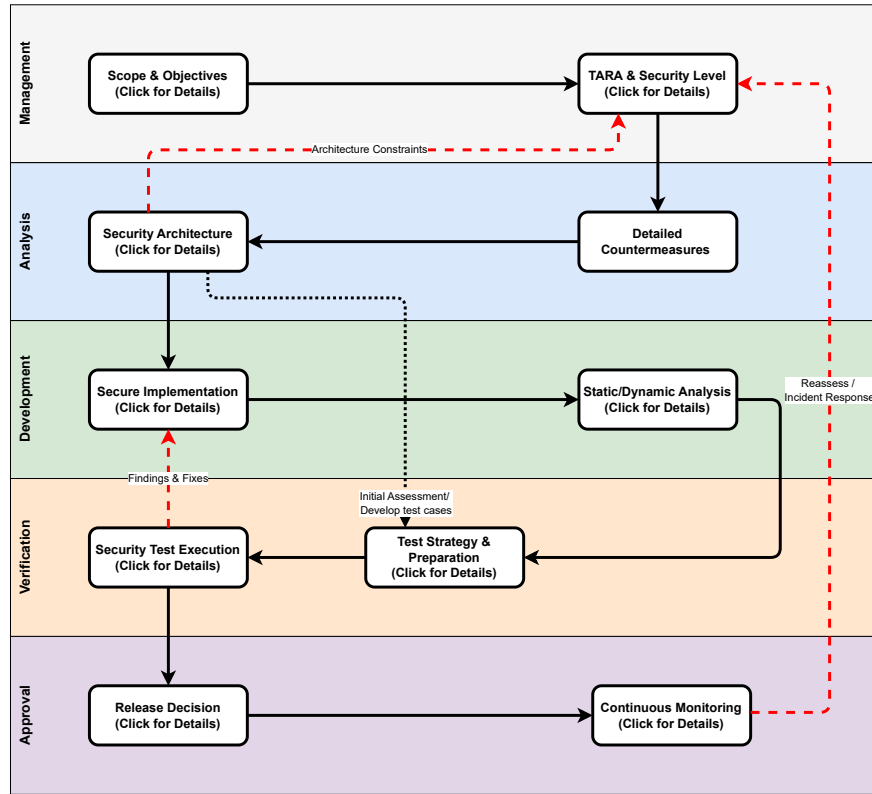


Figure C.1.: High-level overview of the Secure Development Lifecycle (Own illustration)

Responsible, Accountable, Consulted, Informed (RACI) Governance of the Verification Lifecycle To ensure strict organisational accountability, responsibility for the testing phases is governed through a RACI matrix. This management method is described as ensuring exactly one role is accountable for a given activity [77], [78]. In this scheme, the responsible role performs the activity, the accountable role owns and approves the outcome, consulted roles provide input before a decision, and informed roles are notified of the result. Six generic roles are distinguished for this purpose [79, p. 6].

- **Security Engineering:** Holds technical accountability for test scope and execution integrity.
- **System/Requirements Engineering:** Consulted where a requirement or asset boundary is affected.
- **Test Engineering:** Responsible for procedure preparation, execution, and verdict assignment.
- **Software/Firmware Development:** Consulted on findings and tasked with implementing corrective actions.
- **Quality/Compliance Management:** Responsible for coverage review and evidence

governance.

- **Release Authority:** Accountable for the final release decision.

Table C.1 defines the distribution of these responsibilities across the verification phases.

Table C.1.: RACI matrix for the verification lifecycle (Adapted from [77])

Activity	Sec. Eng.	Sys/Req Eng.	Test Eng.	SW/FW Dev.	QA/ Compl.	Release Auth.
Design	A	C	R	I	I	I
Preparation	C	I	A/R	C	I	I
Execution	A	I	R	C	I	I
Evidence/Eval.	C	I	R	C	A	I
Release Decision	I	I	I	I	C	A/R

As illustrated in Table C.1, accountability shifts between phases. It follows the organisational ownership of a phase's primary deliverable rather than a fixed functional assignment [77]. Distinguishing the role responsible for executing a verification activity from the role accountable for its formal closure reflects the practical division of labour documented in secure software engineering [80, p. 5]. Ultimately, these role assignments directly support IEC 62443-4-1 Practice SVV. This standard mandates a rigorous process demonstrating that the product's security functions meet all specified requirements [43, pp. 37].

D. Tool Commands

This appendix lists the tool commands used for the activities executed under the NV1 network position, as reported in Section 4.3. The placeholder `$SRIO` denotes the IP address of the SRIO, `192.168.0.2`. All commands were executed from the Kali Linux test system under NV1 in the isolated laboratory environment described in Section 4.1.2. The commands are reproduced as run, and the file names below are the names of the stored evidence artefacts produced during the test campaign.

```
export SRIO=192.168.0.2
```

The output file names correspond to the stored evidence.

D.1. Attack-Surface Enumeration

The scan was executed on two separate dates to confirm reproducibility, both with the same result (Section 4.3.1). The commands and file names of the later run are given below; the earlier run is stored under the `cm001_srio_*` basename with otherwise identical commands.

```
# Full TCP scan with service detection
sudo nmap -sS -sV -p- -T3 -oA cm_rq001-02_tcp_srio $SRIO
# UDP top-ports scan
sudo nmap -sU --top-ports 200 -oA cm_rq001-02 $SRIO
# Web-server scan
nikto -h http://$SRIO -output cm_rq001-02_srio.html -Format html
```

D.2. Identification-Surface Inventory

```
for p in deviceinfo/hwversion deviceinfo/hwrevision deviceinfo/swrevision \
```

```
deviceinfo/swinfo/cpubootloaderversion deviceinfo/swinfo/scpuversion \  
deviceinfo/swinfo/scpubootloaderversion firmware/version \  
fieldbussetup/fieldbusfirmware; do  
echo "== $p =="  
curl -s http://$SRI0/$p/getdata  
echo  
done | tee cm_rq004-01_inventory.txt
```

D.3. Cleartext Transmission of the Identification Data

```
sslsan --show-certificate $SRI0:80 | tee cm_rq006-03_sslscan.txt
```

D.4. Intervention Evidence and Time Attribution

```
# Read the error log and the system tick counter before and after each event  
curl -s http://$SRI0/devicestatus/errorlog/loglist | tee errorlog.json  
curl -s http://$SRI0/systemtime/systick/getdata | tee systick.txt  
# Manual sequence: baseline; short-circuit pins 1 and 3 of a digital input;  
# remove power for a cold restart;  
induce the second short-circuit;  
repeat the reads.
```

D.5. Availability under Connection Load

```
# Application-layer load test at 50 and at 3 concurrent connections  
siege -c 50 -t 1M -v http://$SRI0/deviceinfo/getdata \  
--log=cmrq010_siege.log  
siege -c 3 -t 1M -v http://$SRI0/deviceinfo/getdata \  
--log=cmrq010_3connections_siege.log  
# Connection-hold procedure: occupy the available HTTP slots, then issue  
bash  
export SRI0=192.168.0.2  
echo " Baseline: Connection working normally"  
| tee cm_rq007-03_00_baseline.txt  
( time curl -s --max-time 5 http://$SRI0/deviceinfo/swrevision/getdata )
```

```
>> cm_rq007-03_00_baseline.txt 2>&1
cat cm_rq007-03_00_baseline.txt

echo "Block both slots"
for i in 1 2; do
(exec 3<>/dev/tcp/$SRIO/80; printf 'GET /deviceinfo HTTP/1.1\r\nHost:
x\r\n\r\n' >&3; sleep 600) &
done
jobs | tee cm_rq007-03_01_jobs.txt

echo "3rd attempt while both slots are occupied (expected: timeout)"
| tee cm_rq007-03_02_blocked.txt
( time curl -s --max-time 5 http://$SRIO/deviceinfo/swrevision/getdata )
>> cm_rq007-03_02_blocked.txt 2>&1
cat cm_rq007-03_02_blocked.txt

echo "Free up one slot"
kill %1
echo "Retry after freeing up slot (expected: successful again)"
| tee cm_rq007-03_03_after_release.txt
( time curl -s --max-time 5 http://$SRIO/deviceinfo/swrevision/getdata )
>> cm_rq007-03_03_after_release.txt 2>&1
cat cm_rq007-03_03_after_release.txt
```

D.6. Layer-Two and Network Flooding

```
sudo hping3 --flood --rand-source $SRIO
sudo macof -i eth1
```

D.7. Robustness under Malformed Input

```
# boofuzz fuzzing script
# File: cm010_fuzz.py
from boofuzz import *

session = Session(target=Target(connection=TCPSocketConnection("$SRIO", 80)))
s_initialize("request")
```

```
s_string("USER")          # field to fuzz/ "GET"
s_delim(" ")
s_string("value")
session.connect(s_get("request"))
session.fuzz()

# Run it:
python3 cm010_fuzz.py
```

D.8. Access Control and Configuration Interfaces

```
# Write probe against safety-relevant IoT-Core nodes
curl -s -X POST http://$SRIO/safecom/f_address/setdata \
-d '{"value":123}' -w "%{http_code}\n"
curl -s -X POST http://$SRIO/io/do/port7/pin4/digital_output/setdata \
-d '{"value":true}' -w "%{http_code}\n"
curl -s -X POST http://$SRIO/devicetag/applicationtag/setdata \
-d '"factoryXYZ"' -w "%{http_code}\n"
# Active web scan through a local proxy
# Disabling the API key is acceptable only in the isolated laboratory.
zapproxy -daemon -host 127.0.0.1 -port 8090 -config api.disablekey=true
curl "http://127.0.0.1:8090/JSON/spider/action/scan/?url=http://$SRIO/"
```